Open Source Game Theory: Formalization, Transparency, and Cooperation in Program Games

COLOMBAN DUCLAUX, ETH Zürich, Switzerland and EuroSafeAI, Switzerland
RICCARDO FORMENTI, ETH Zürich, Switzerland and EuroSafeAI, Switzerland
PEPIJN COBBEN, ETH Zürich, Switzerland and EuroSafeAI, Switzerland
TERRY JINGCHEN ZHANG, University of Oxford, United Kingdom, EuroSafeAI, Switzerland, and Vector Institute, Canada
BERNHARD SCHÖLKOPF, ETH Zürich, Switzerland, Max Planck Institute for Intelligent Systems, Germany, and ELLIS Institute, Germany
ZHIJING JIN, EuroSafeAI, Switzerland, Max Planck Institute for Intelligent Systems, Germany, University of Toronto, Canada, and Vector Institute, Canada

Abstract. Cooperation among AI agents can undermine alignment protocols when it takes the form of collusion. We develop formal foundations for proof-based open-source game theory, in which programs condition their actions on proofs about their opponents. We specify a concrete proof system, resolve an open problem concerning DupocBot and CupodBot through a transposition argument, and formalize the framework in Lean 4. We then extend the framework to partial transparency, allowing programs to receive beliefs about their opponents instead of exact source code. This formulation connects fully transparent program games with the underlying game under no transparency and identifies conditions under which their equilibrium outcomes coincide. In Prisoner's Dilemma experiments, increasing transparency increases both the cooperation sustainable in equilibrium and the cooperation selected by replicator dynamics in the settings studied. Together, these contributions provide a machine-checkable foundation for analyzing strategic interactions and a framework for studying how transparency enables cooperation, with implications for collusion and AI alignment.

1 Introduction

Artificial intelligence systems increasingly act autonomously and interact with one another, both during training and deployment (Hammond et al. 2025). Yet these strategic interactions can be puzzling; for example, reports on the recent OpenAI-Hugging Face incidents describe AI agents *collaborating* towards a shared goal – tricking the evaluator – even when doing so appeared *individually irrational*, such as when agents ended their own runs to obtain information for others (METR 2026; OpenAI 2026). The classical prediction (and recommendation) of game theory is that rational players never choose dominated actions, making them defect in the Prisoner's Dilemma although cooperation would be mutually beneficial. Moreover, even when cooperative equilibria exist, as in the infinitely repeated Prisoner's Dilemma, one must still explain which mechanisms of equilibrium selection lead players to coordinate on them. The agents' behavior thus raises questions about the *"unreasonable effectiveness of collaboration"* (one might say collusion) among AI models. Understanding how such collaboration emerges and which conditions sustain it will be essential to aligning superintelligent models that interact strategically towards one another and towards humans.

One instance of such cooperation that game theory can explain arises in *program meta-games* (Kalai et al. 2010; Tennenholtz 2004), introduced to model commitments among human institutions and software programs. In these

Authors' Contact Information: Colomban Duclaux, cduclaux@ethz.ch, ETH Zürich, Zürich, Switzerland and EuroSafeAI, Zürich, Switzerland; Riccardo Formenti, rformenti@ethz.ch, ETH Zürich, Zürich, Switzerland and EuroSafeAI, Zürich, Switzerland; Pepijn Cobben, ETH Zürich, Zürich, Switzerland and EuroSafeAI, Zürich, Switzerland; Terry Jingchen Zhang, University of Oxford, Oxford, United Kingdom and EuroSafeAI, Zürich, Switzerland and Vector Institute, Toronto, Canada; Bernhard Schölkopf, ETH Zürich, Zürich, Switzerland and Max Planck Institute for Intelligent Systems, Tübingen, Germany and ELLIS Institute, Tübingen, Germany; Zhijing Jin, EuroSafeAI, Zürich, Switzerland and Max Planck Institute for Intelligent Systems, Tübingen, Germany and University of Toronto, Toronto, Canada and Vector Institute, Toronto, Canada.

games, players' actions are implemented by mutually transparent commitment devices, which we call *programs*. These devices may take the form of software code, company bylaws, or governmental constitutions, and mutual transparency means that each program has access to the internal workings of the other players' programs. The Nash equilibria of the resulting meta-game, called *program equilibria*, are shown to sustain outcomes that are not equilibria of the base game (Tennenholtz 2004), including cooperation in the Prisoner's Dilemma. Additionally, programs can prove statements about their opponents within a given formal system, and thus condition their behavior on the resulting proofs (Bárász et al. 2014; Critch et al. 2022), making cooperation robust. We follow the latter in referring to this field as open source game theory[1].

We recognize that extending this analysis to more complex and realistic games will require increasingly automated proof search. To this end, we develop a framework for open source game theory and place it on formal, machine-checkable foundations. We begin by **(i)** specifying the formal proof system in which programs prove statements about one another (Section 3). Using a transposition argument, we **(ii)** prove the outcome of `DupocBot` against `CupodBot` (Section 3.3), resolving an open problem in the literature. We then **(iii)** apply related techniques to other questions left open by Critch et al. (2022) (Appendix B.3). Finally, we **(iv)** formalize our framework in Lean 4 (Moura and Ullrich 2021), so that each outcome we prove is a theorem checked by the Lean kernel (Section 4). Lean allows generated proofs to be checked independently, providing verifiable feedback for proof search and a potential source of verifiable rewards for training future proof agents.

Our motivating application is AI alignment. Training protocols (e.g. debate (Irving et al. 2018)) and deployment protocols (e.g. oversight (Greenblatt et al. 2024)) often rely on agents challenging or evaluating one another. If those agents collude and pursue objectives that conflict with human preferences, the effectiveness of the above methods will be undermined[2]. Open-source game theory is well suited to modelling interactions in which agents condition their behaviour on information about one another. Its classical formulation, however, assumes full transparency, in that each program receives its opponent's exact program as input. In most settings of practical interest, however, players have only partial access to one another; for instance, a deployed model may be queried as a black box without exposing its weights, or agents may know little more than that they belong to the same base model family or company. We therefore extend the framework to partial transparency and examine its implications for cooperation. We first **(v)** introduce a model of partial transparency in which each program receives as input a belief about its opponent's program, determined by the actual program but not necessarily identifying it uniquely (Section 2.2), and **(vi)** apply this framework to our running example of the Prisoner's Dilemma (Section 2.2.1). We then **(vii)** show how the framework recovers both open source game theory under full transparency and the base game under no transparency, and identify conditions under which its equilibrium outcomes coincide with those of the base game (Section 2.2.2). We **(viii)** show through examples that program meta-games based on the Prisoner's Dilemma admit cooperative equilibria absent from the underlying game, and that replicator dynamics can select these equilibria under both full transparency (Section 5.1) and partial transparency (Section 5.2). More importantly, we **(ix)** show experimentally that increasing transparency increases both the cooperation sustainable in equilibrium and the cooperation selected by replicator dynamics (Section 5.2.2).

2 Program Equilibrium under Full and Partial Transparency

The classical framework of open-source game theory (Section 2.1) assumes full transparency, in that each program receives the opponent's exact program as input. This is a strong descriptive assumption. In most settings of

[1]The name refers to open-source software, i.e. software whose source code is widely available for inspection and modification, which was the motivating example in the seminal work of Tennenholtz (2004).

[2]By collusion we mean that the agents coordinate on an outcome that benefits them with respect to their own, possibly misaligned, preferences but that is contrary to the purpose of the protocol. When each agent is individually tempted to report, the interaction is a Prisoner's Dilemma and collusion is mutual cooperation.

practical interest, players have only partial access to one another; for instance, a deployed model may be queried as a black box without exposing its weights, or agents may know little more than that they belong to the same base model family or company. Partial transparency of this kind motivates the diff meta games of Oesterheld et al. (2023). In these games, each program observes a single scalar measure of similarity between itself and the opponent's program and maps this signal to a mixed action. More importantly, the literature treats transparency as an exogenous feature of the environment. Instead, we adopt here the perspective a designer who controls the players' informational environment, that is, controls what each program may observe about its opponent, in the spirit of information design (Bergemann and Morris 2019). These observations, together with our guiding example of AI alignment, motivates studying partial transparency as a design variable in proof-based OSGT. We develop the general model in Section 2.2 and use as an example the Prisoner's Dilemma in Section 2.2.1.

2.1 Open Source Game Theory

A game G in normal form is a tuple $G = (I, (A_i, u_i)_{i \in I})$, where I is a finite set of players, A_i is the finite set of actions available to player i, and $u_i : \prod_{j \in I} A_j \to \mathbb{R}$ is the utility (payoff) function of player i. For simplicity's sake, we restrict our attention to two players games with common action sets, i.e. $I = \{1, 2\}$ and $A \coloneqq A_1 = A_2$. As is customary, for player $i \in I$, let's denote by $-i$ the other player and by $\boldsymbol{a} \in A \times A$ an action profile. Finally, a mixed action for player $i \in I$ is a probability distribution over $A_i = A$, an element of the probability simplex $\Delta(A)$; we extend u_i to $\Delta(A) \times \Delta(A)$ by expectation in the usual way. The (in)famous Prisoner's Dilemma will serve as the main example throughout.

Example 2.1 (Prisoner's Dilemma). The Prisoner's Dilemma is a two-player game G_{PD} with common action set $A = \{C, D\}$, cooperate (C) and defect (D), and payoffs

	C	D
C	(R, R)	(S, T)
D	(T, S)	(P, P)

$$T > R > P > S,$$

where rows are the actions of player 1, columns those of player 2, and each entry lists (u_1, u_2). Since $T > R$ and $P > S$, the action D strictly dominates C for each player. Hence (D, D) is the unique Nash equilibrium[3] of G_{PD}, even though $R > P$ means it is Pareto-dominated[4] by (C, C).

Example 2.1 illustrates the strategic tension between individual incentives and collective outcomes: although mutual cooperation would leave both players better off than mutual defection, each has an incentive to defect regardless of the other's action. Open-source game theory studies a different strategic setting, in which players commit to programs that can condition their actions on their opponents' programs. In this sense, a player's choice can affect what her opponent does. Executing these programs produces an action profile. In particular, a player's program may condition its action on an inspection of the opponents' programs, and conditional commitments of the form "cooperate if and only if I can verify that you will cooperate" become well-defined objects of strategic choice. This setting was first introduced by Tennenholtz (2004), further studied in Kalai et al. (2010) and Bárász et al. (2014), and later termed Open Source Game Theory (OSGT) by Critch et al. (2022).

To make precise the meta-game in which players commit to a program instead of directly playing an action, we build on the formalization and notation of Duclaux, Formenti, Cobben, Schölkopf, et al. (2026). For each player $i \in I$, denote by Σ_i the finite set of admissible programs. Again, we assume common program sets, that is, $\Sigma \coloneqq \Sigma_1 = \Sigma_2$, and we denote by $\boldsymbol{\pi} = (\pi_1, \pi_2) \in \Sigma \times \Sigma$ a program profile. Each program receives its opponent's

[3] A profile $\boldsymbol{a}^* \in A \times A$ is a Nash equilibrium if for all $i \in I$, all $a_i \in A$, $u_i(a_i^*, a_{-i}^*) \geq u_i(a_i, a_{-i}^*)$, that is, no player can increase their payoff by deviating unilaterally.

[4] For action profiles $\boldsymbol{a}$ and $\boldsymbol{a}' \in A \times A$, $\boldsymbol{a}$ is said to Pareto-dominate $\boldsymbol{a}'$ if for all i in I, $u_i(\boldsymbol{a}) \geq u_i(\boldsymbol{a}')$, with strict inequality for some $j \in I$. We say that $\boldsymbol{a}$ is Pareto-optimal if no other action profile Pareto-dominates it.

program as input.[5] We deliberately leave admissibility under-specified at this point, with the only requirement being that every admissible program, run on any admissible program (including itself), halts and returns an action. Section 4.1.1 introduces the concrete language in which the programs of Σ are written, and Figure 2 fixes the finite program libraries used in our experiments.

Definition 2.2 (Mutually halting program set). A common program set Σ is mutually halting if, for all $\pi, \pi' \in \Sigma$, the execution of π on input π' terminates and outputs an action in A. In this case, the execution function $\mathrm{run}\colon \Sigma \times \Sigma \to A$ maps (π, π') to that output, which we write suggestively as $\mathrm{run}(\pi \parallel \pi')$.

Moreover, we assume that every constant program is admissible, that is, for each action $a \in A$, the program that ignores its input and always plays a belongs to Σ. With an abuse of notation, we denote this program by a, so that $\mathrm{run}(a \| \pi) = a$ for all $\pi \in \Sigma$, and we define the natural embedding $A \subseteq \Sigma$. This assumption and the mutually halting condition of Definition 2.2 will be assumed throughout without further mention.

The execution function run induces the *outcome map* of the game, which to a profile of programs assigns its played action profile

$$\mathrm{outcome} : \Sigma \times \Sigma \to A \times A, \quad \mathrm{outcome}(\boldsymbol{\pi}) \coloneqq \Big(\mathrm{run}(\pi_1 \| \pi_2), \mathrm{run}(\pi_2 \| \pi_1)\Big),$$

which assigns to each program profile $\boldsymbol{\pi} = (\pi_1, \pi_2)$ the action profile it produces. Each player utility is then naturally extended to program profiles,

$$U_i : \Sigma \times \Sigma \to \mathbb{R}, \qquad U_i(\boldsymbol{\pi}) \coloneqq u_i\big(\mathrm{outcome}(\boldsymbol{\pi})\big).$$

Since $A \subseteq \Sigma$ and constant programs play their constant action against any opponent, the meta-utility U_i indeed restricts to u_i on $A \times A$. The tuple $G^{\mathrm{prog}} := \big(I, (\Sigma_i, U_i)_{i \in I}\big)$ constitutes a normal form game, which we call the program meta-game associated with G. We are ready to give the definition of equilibrium introduced by Tennenholtz (2004) for program meta-games.

Definition 2.3 (Program Equilibrium). A profile $\boldsymbol{\pi}^* \in \Sigma \times \Sigma$ is a Program Equilibrium (PE) of G if it is a (pure) Nash equilibrium of G^{prog}, i.e., for every $i \in I$ and every $\pi_i' \in \Sigma$,

$$U_i(\pi_i^*, \pi_{-i}^*) \;\geq\; U_i(\pi_i', \pi_{-i}^*).$$

More generally, a profile $\boldsymbol{\alpha}^* \in \Delta(\Sigma) \times \Delta(\Sigma)$ is a mixed Program equilibrium if it is a mixed Nash equilibrium of G^{prog}, where we have extended U_i by expectation.

We first observe that if the underlying game G admits a pure Nash equilibrium, then so does the meta-game G^{prog}. In fact, it suffices to replace each equilibrium action by the corresponding constant program. All proofs for this Section are deferred to Appendix A.

Theorem 2.4 (Pure Nash equilibria are program equilibria). *Let $\boldsymbol{a}^* \in A \times A$ be a pure Nash equilibrium of G. Then the profile of constant programs $\boldsymbol{a}^* \in \Sigma \times \Sigma$ is a pure program equilibrium of G.*

Existence does not depend on G having a pure equilibrium. Since Σ is finite and nonempty (it contains A), the meta-game G^{prog} is a finite normal-form game, so by Nash's existence theorem (Nash 1950, 1951) it admits a mixed Nash equilibrium $\boldsymbol{\alpha}^* \in \Delta(\Sigma) \times \Delta(\Sigma)$. By definition, $\boldsymbol{\alpha}^*$ is a mixed program equilibrium of G.

Example 2.5 (Prisoner's Dilemma, cont'd). By Theorem 2.4, mutual defection (D, D) remains a program equilibrium of G_{PD}. Unlike in the Prisoner's Dilemma, however, it is no longer the unique equilibrium outcome, as Program equilibria can also sustain cooperation. The simplest example is Tennenholtz's `CliqueBot` (Tennenholtz

[5] Throughout, we fix a computable injective encoding of finite program descriptions into $\mathbb{N}$ and, when no confusion arises, identify each program with its code.

2004), which cooperates if its input is syntactically identical to its own code and defects otherwise. If `CliqueBot` is admissible, then the profile (`CliqueBot`, `CliqueBot`) yields (C, C). Any unilateral deviation π' obtains defection, so it earns $\max\{S, P\} = P < R$. The proof-based bots introduced by Critch et al. (2022) show that cooperation can also be achieved by programs robust to syntactic differences, such as `DupocBot`, which we recall below.

In this paper we focus our attention on proof-based OSGT (Bárász et al. 2014; Critch et al. 2022), where programs will have access to a formal proof system S (such as Peano Arithmetic) in which to state hypotheses about their opponents and search for proofs of them[6]. We fix S in section 3.1; until then, all we need is a proof search function that searches for a proof of a statement φ in S depending on a program $\pi' \in \Sigma$, of length at most $k \in \mathbb{N}$.

```
def proof_search(k, π′, φ):
    # Returns True iff S has a proof of φ about π′ of length at most k.
```

By construction, `proof_search`(k, π', φ) returns `True` exactly when S has a proof of φ of length at most k. We require throughout that S be sound, i.e., whatever S proves is true. This implies that if the search succeeds, then φ holds, and, in particular, π' actually does what φ says it does (see Section 3.2 for more on this).

Example 2.6 (Prisoner's Dilemma, cont'd). Consider $G_{\text{PD}}^{\text{prog}}$, the program meta-game induced by the Prisoner's Dilemma. The canonical example of a provability-based program is `DupocBot`(k), for $k \in \mathbb{N}$, "Defect Unless Proof of Cooperation", which on an opponent $\pi \in \Sigma$,

```
def DupocBot(k)(π):
   if proof_search(k, π, run(π‖DupocBot(k)) = C):
        return C
   else:
        return D
```

The interest of this program lies in the fact that it cooperates with cooperative opponents, but never gets exploited[7], i.e. never cooperates when the other defects. The difficulty in proof-based OSGT lies in the logical and strategic interactions between programs that prove propositions about other programs. As an example, consider the following:

$$\text{outcome}(\texttt{DupocBot}(k), \texttt{DupocBot}(k))$$

It is not immediately clear what the result should be. Both programs are identical, and try to prove that their opponent cooperates. As Critch et al. (2022) put it, we face am apparent circular dependency: "the only way `DupocBot`(k) returns C is if it first finds a proof in fewer than k characters that `DupocBot`(k) returns C!" Intuitively, one might argue that for cooperation to occur, `DupocBot`(k) would have to prove that `DupocBot`(k) proves that `DupocBot`(k) proves that... and such a proof could not exist in finite length k. So one might expect that outcome(`DupocBot`(k), `DupocBot`(k)) $= (D, D)$. Quite surprisingly, this is not the case; Critch et al. (2022) show that for large k the outcome is in fact mutual cooperation (C, C), using an argument via the Parametric Bounded Löb Theorem (PBLT) (Critch 2019, Theorem 4.2). For such k, assuming `DupocBot`$(k) \in \Sigma$, the profile (`DupocBot`(k), `DupocBot`(k)) is a program equilibrium; in fact, each player receives R, and soundness of S (hence unexploitability of `DupocBot`) rules out unilateral deviations. Thus mutual cooperation is sustained as an equilibrium outcome

[6] Another paradigm of OSGT is simulation-based OSGT, where programs simulate their opponent instead of formulating mathematical proofs (Cooper et al. 2025; Kovarík et al. 2023; Oesterheld 2019). Simulation and provability are two sides of the same coin, as a successful simulation is a concrete computational proof of a simulated program's behavior. Our library of programs thus allows for both kinds of bots. However, naive mutual simulation can lead to infinite recursion, and thus we focus this paper on proof-based programs.

[7] This is a simple consequence of soundness of the formal proof system S (see Section B.1).

of the Prisoner's Dilemma. In this way, the promise of OSGT is fulfilled, with better payoffs for both players through transparency.

It is important to note how transparency of the opponent's source code can support program equilibria whose played actions, such as mutual cooperation in the Prisoner's Dilemma (see Example 2.6), cannot be supported in equilibrium in the underlying game. By conditioning cooperation on verifiable properties of the opponent's program, players can make cooperation robust to unilateral deviations.

2.2 Partial Transparency

As in Section 2.1, we fix a finite base game G and a finite set Σ of admissible programs, shared by both players and common knowledge between them. Now, however, a program no longer receives the opponent's program as input. Instead, it receives a probability distribution $\mu \in \Delta(\Sigma)$, interpreted as the program's belief about which program the opponent has actually submitted. We therefore extend Definition 2.2 to allow beliefs as inputs.

Definition 2.7 (Mutually halting program set; belief inputs). A common program set Σ is mutually halting if, for all $\pi \in \Sigma$ and all beliefs $\mu \in \Delta(\Sigma)$, the execution of π on input μ terminates and outputs an action in A. In this case, the execution function $\mathrm{run}\colon \Sigma \times \Delta(\Sigma) \to A$ maps (π, μ) to that output, which we write suggestively as $\mathrm{run}(\pi \parallel \mu)$.

As in Section 2.1, we assume that every constant program is admissible, that is, for each action $a \in A$, the program that ignores its input belief and always outputs a belongs to Σ. We again denote this program by a, so that $\mathrm{run}(a \| \mu) = a$ for every $\mu \in \Delta(\Sigma)$, and we retain the embedding $A \subseteq \Sigma$.

The information structure of the game[8] is described by a pair of *transparency maps* $\tau = (\tau_1, \tau_2)$, where $\tau_i\colon \Sigma \to \Delta(\Sigma)$ for each $i \in I$. Here $\tau_i(\pi)$ is the belief that player i's program receives as input when the opponent $-i$ submits π. Intuitively, τ_i describes what player i's program can learn about its opponent, for instance through black-box queries or knowledge of a shared model family, and it is the object the designer controls. At one extreme, $\tau_i(\pi) = \delta_\pi$ recovers the full transparency of Section 2.1; at the other, a constant τ_i reveals nothing, and player i effectively chooses an action of the base game G; for more details, see Section 2.2.2.

Naturally extending the definitions of Section 2.1, we define the outcome map induced by the execution function run and the transparency maps τ,

$$\mathrm{outcome}^\tau : \Sigma \times \Sigma \to A \times A, \quad \mathrm{outcome}^\tau(\boldsymbol{\pi}) \coloneqq \Big(\mathrm{run}(\pi_1 \| \tau_1(\pi_2)), \mathrm{run}(\pi_2 \| \tau_2(\pi_1))\Big),$$

which assigns to each program profile $\boldsymbol{\pi} = (\pi_1, \pi_2)$ the action profile it produces. The only change from Section 2.1 is that each program now runs on the belief it receives from the opponent's code. Each player's utility is then extended to program profiles by

$$U_i^\tau : \Sigma \times \Sigma \to \mathbb{R}, \qquad U_i^\tau(\boldsymbol{\pi}) \coloneqq u_i\big(\mathrm{outcome}^\tau(\boldsymbol{\pi})\big).$$

The tuple $G^\tau \coloneqq \big(I, (\Sigma_i, U_i^\tau)_{i \in I}\big)$ is a normal-form game, which we call the *τ-program meta-game* associated with G; the superscript makes its dependence on the transparency maps τ explicit. Following Definition 2.3, a profile $\boldsymbol{\pi} \in \Sigma \times \Sigma$ is a τ-Program Equilibrium (or simply a Program Equilibrium, when τ is clear from context) if it is a pure Nash equilibrium of G^τ; mixed τ-Program Equilibria are defined analogously.

Definition 2.8 (τ-Program Equilibrium). A profile $\boldsymbol{\pi}^* \in \Sigma \times \Sigma$ is a τ-Program Equilibrium (PE) of G if it is a (pure) Nash equilibrium of G^τ, i.e., for every $i \in I$ and every $\pi_i' \in \Sigma$,

$$U_i^\tau(\pi_i^*, \pi_{-i}^*) \;\geq\; U_i^\tau(\pi_i', \pi_{-i}^*).$$

[8] A different idea for modeling partial information, or feedback, in games is the self-confirming equilibrium of Battigalli (1987) and Fudenberg and Levine (1993).

Theorem 2.9 (Pure Nash equilibria are τ-program equilibria). *Let τ be any pair of transparency maps and let $\boldsymbol{a}^* \in A \times A$. Then the profile of constant programs $\boldsymbol{a}^* \in \Sigma \times \Sigma$ is a pure τ-Program Equilibrium if and only if $\boldsymbol{a}^*$ is a pure Nash equilibrium of G. In particular, every pure Nash equilibrium of G is a τ-Program Equilibrium for every choice of τ.*

More generally, the existence of τ-Program Equilibria does not depend on G having a pure equilibrium, as one can apply Nash's existence theorem (Nash 1950, 1951) to G^τ to prove existence of a mixed τ-Program Equilibrium of G.

2.2.1 An Example in the Prisoner's Dilemma. We now return to our running example, the Prisoner's Dilemma (Examples 2.1 and 2.6), and illustrate partial transparency through concrete transparency maps and explicit programs. We assume throughout a common transparency map $\tau_1 = \tau_2$ for the two players; write τ for it. Explicit maps are found in section F.1.1.

Given a program `Bot` of the open source Prisoner's Dilemma $G_{\text{PD}}^{\text{prog}}$, we can extend it to a program `TauBot` of the meta-game G_{PD}^{τ}, which takes a belief $\mu \in \Delta(\Sigma)$ as input. In particular, `TauBot` runs the decision procedure of `Bot` against each program in the support of μ and aggregates the resulting actions. The simplest aggregator is a threshold rule, where for $\alpha \in [0, 1]$, cooperate if and only if the μ-mass of the programs against which `Bot` would cooperate is at least α; we call α the caution parameter. We give below the example of `TauDupoc`(k, α).

```
def TauDupocBot(k, α)(μ):
    mass = 0
    for π' in supp(μ):
        if proof_search(k, π', run(π' ‖ δ_TauDupocBot(k,α)) = C):
            mass = mass + μ(π')
    if mass ≥ α:
        return C
    else:
        return D
```

Unlike `DupocBot`(k), `TauDupocBot`(k, α) is no longer unexploitable. Soundness of S still guarantees that every program counted in `mass` cooperates on the point-mass input $\delta_{\texttt{TauDupocBot}(k,\alpha)}$, but for poorly behaved transparency map τ, the actual opponent need not be one of these programs.

The threshold rule given above is linear in μ and therefore admits an expected utility representation[9]; non-linear aggregators, such as the maximum or minimum or a variational functional, capture ambiguity-sensitive behaviour (Gilboa and Marinacci 2016). We do not explore their properties here.

2.2.2 When Does Transparency Matter? Choosing a program rather than an action can alter incentives because a change in one player's program may induce a change in her opponent's response. Suppose, however, that a player can change what her program does without changing what her opponent observes. In the Prisoner's Dilemma, for example, replacing a cooperative program with an indistinguishable program that defects would leave the opponent's cooperation unchanged, making the deviation profitable. We say that a player has *undetectable deviations* if, for every program she might submit and every input belief it might receive, she can obtain any desired action by substituting a program that her opponent cannot distinguish from the original. Against a fixed opponent's program, she can therefore change her own action while holding the opponent's action fixed, just as in the base game. In this sense, rational behaviour in the meta game G^τ can differ from rational behaviour in the base game G only insofar as τ reveals something about how programs behave.

[9]This holds on the state space Σ and the larger consequence space $\{C, D\} \times \{0, 1\}$, and let playing a against π give the consequence $(a, f(\pi))$, where $f(\pi) = 1$ if `TauBot` cooperates on the point-mass input δ_π and $f(\pi) = 0$ otherwise.

Definition 2.10 (Undetectable Deviations). Player $i \in I$ has undetectable deviations if for every $\pi \in \Sigma$, every $\mu \in \tau_i(\Sigma)$, and every $x \in A$ there exists a $\rho \in \Sigma$ such that

(1) ρ is indistinguishable from π to player $-i$, that is, $\tau_{-i}(\rho) = \tau_{-i}(\pi)$;
(2) ρ plays x on input μ, that is, $\mathrm{run}(\rho \| \mu) = x$.

A program of player $-i$ can condition its action only on the belief it receives, $\tau_{-i}(\pi_i)$, and can therefore reward or punish player the opponent i only through what is revealed by this belief. f i has undetectable deviations, any arbitrary change in i's behaviour can be made without changing what $-i$ observes. Hence no program of $-i$ can make its response depend on what i actually does[10]; as Theorem 2.11 shows, this implies that the program meta-game reduces to the base game. This idea is the analogue of the informativeness principle in moral hazard (Holmström 1979) where an additional signal can improve the optimal contract if and only if it provides information about the agent's action beyond what is already observed in the outcome.

Theorem 2.11. *Assume both players have undetectable deviations. Then the outcomes of pure τ-program equilibria are exactly the pure Nash equilibria of G.*

The two extreme transparency structures illustrate the result. If τ_{-i} is constant, so that player $-i$ observes nothing about i's program, then player i trivially has undetectable deviations (using the x-constant program). When τ is constant, Theorem 2.11 shows that the outcomes of pure τ-PE are exactly the pure Nash equilibria of G; in this case, each player effectively chooses an action of the base game. On the other hand, full transparency, $\tau_i(\pi) = \delta_\pi$, recovers the open-source setting of Section 2.1. There neither player has undetectable deviations[11], and program equilibria can sustain outcomes that are not Nash equilibria of G, such as mutual cooperation in the Prisoner's Dilemma (Example 2.6).

By varying the transparency map τ, we thus obtain a family of games ranging from the base game G to the fully transparent meta-game G^{prog} of Tennenholtz (2004). At one extreme, programs receive no information about their opponents; at the other, they can condition their behaviour on the opponent's exact code. Throughout this family, Theorem 2.9 shows that every pure Nash equilibrium of G remains a program equilibrium, giving the inclusion NE $\subseteq$ PE. Whether the sets of program equilibria, or their induced outcomes, vary monotonically in τ under a suitable order on transparency maps is a further question we do not approach here. Greater transparency may allow programs to detect and respond to deviations that would otherwise go unnoticed, but it also changes the inputs they receive and hence potentially their behaviour. It is therefore not immediate that outcomes sustained under one transparency map remain sustainable under a more informative one. For this exercise, the results of Battigalli et al. (2016) and Formenti (2026) may be useful.

3 Outcomes under the Formal System S

In simulation-based OSGT (Cooper et al. 2025; Kovarík et al. 2023; Oesterheld 2019), a program decides what to play based on running its opponent, and no logic is involved. In proof-based OSGT, outcomes depend on which statements S proves within a given length, and on the actual proofs themselves. In this section we fix the

[10]Black-box testing provides a concrete example. If player $-i$ observes only how player i's program behaves on a set of test inputs, then i can deviate undetectably at every input outside this set. Testing can therefore deter opportunistic behavior only if the inputs a program faces in the actual interaction cannot be distinguished from test inputs. This the concern we face with evaluation awareness in AI systems.

[11]Definition 2.10 imposes a simple counting constraint. Fix any $\mu \in \tau_i(\Sigma)$ and any $\pi \in \Sigma$. Applying the definition to (π, μ, x) for each $x \in A$ yields programs ρ_x with $\tau_{-i}(\rho_x) = \tau_{-i}(\pi)$ and $\mathrm{run}(\rho_x \,\|\, \mu) = x$, which are distinct for distinct x. Hence every set of programs that player $-i$ cannot distinguish from one another contains at least $|A|$ elements. Since these sets partition Σ and there are $|\tau_{-i}(\Sigma)|$ of them,

$$|\tau_{-i}(\Sigma)| \cdot |A| \le |\Sigma|.$$

In particular, if τ_{-i} is injective, as under full transparency $\tau_{-i}(\pi) = \delta_\pi$, player i does not have undetectable deviations as soon as $|A| \ge 2$. This is consistent with Section 2.1, where program equilibria can induce outcomes that are not Nash equilibria of G (Example 2.6).

formal system S (section 3.1) and show how we determine outcomes under it (section 3.2). Finally, Section 3.3 introduces a new one, transposition, and uses it to settle Open Problem 3 of Critch et al. (2022) for S: the outcome of `DupocBot` against `CupodBot`. We work in the Prisoner's Dilemma throughout.

3.1 The Formal System S

We now fix the formal system S in which programs search for proofs about their opponents' behavior. Critch et al. (2022, Appendix B) work with an unspecified extension of Peano Arithmetic (PA) that is only required to satisfy a list of efficiency assumptions. We instead fix a concrete system, which we later formalize in Lean 4 (Moura and Ullrich 2021) in Section 4.1.3.

Definition 3.1 (The proof system S). The proof system S consists of the following three components.

(S1) *Language.* $\mathcal{L}_{\text{Act}}$ is the language of arithmetic $\{0, 1, +, \cdot, <\}$ extended by two constant symbols c_C and c_D, one per action cooperate C and defect D;

(S2) *Theory.* T_{Act} consists of Peano Arithmetic together with the action axiom

$$(c_C = 0 \wedge c_D = 1) \ \vee \ (c_C = 1 \wedge c_D = 0),$$

stated together with its transpose, obtained by simultaneously exchanging c_C and c_D[12];

(S3) *Proofs.* For a sentence φ of $\mathcal{L}_{\text{Act}}$, a proof of φ is a derivation of φ from T_{Act} in a standard first-order calculus. Its length is the number of symbols it contains. We write $\vdash \varphi$ if φ has a proof, and $\vdash_k \varphi$ if it has a proof of length at most $k \in \mathbb{N}$.

The action axiom, together with its transpose, is the only axiom of T_{Act} beyond those of PA. It states that c_C and c_D denote the numbers 0 and 1, in one of the two possible orders, without specifying which. Consequently, in every model $\mathcal{M}$ of T_{Act}, the pair $(c_C^{\mathcal{M}}, c_D^{\mathcal{M}})$ is either $(0^{\mathcal{M}}, 1^{\mathcal{M}})$, which we call the *standard reading*, or $(1^{\mathcal{M}}, 0^{\mathcal{M}})$, the *swapped reading*. Conversely, every model of PA becomes a model of T_{Act} under either reading of the two constants. Hence the models of T_{Act} are exactly the models of PA equipped with one of the two readings.

A delicate point, familiar to readers versed in logic, deserves discussion. This paper uses two distinct systems to prove statements. The first is S, the formal proof system used by the programs; a call to `proof_search`$(k, \cdot, \varphi)$ determines whether φ has a proof in S of length at most k. The second is the metatheory, the system in which *we* reason in, "ordinary" mathematics in this Section and Lean in Section 4.1.3. Table 1 collects the notation we use for the rest of the paper. We write $\Box_k\varphi$ for the arithmetical sentence asserting that some number codes a proof of φ in S of length at most k. By contrast, $\vdash_k \varphi$ denotes the assertion, made in the metatheory, that such a proof exists. These are related by $\vdash_k \varphi \iff \mathbb{N} \models \Box_k\varphi$.[13] Thus, $\Box_k\varphi$ expresses bounded provability within the language of S, whereas $\vdash_k \varphi$ asserts bounded provability in our discussion of S. We give an example of three statements that must not be confused; $\nvdash_k \varphi$ says that φ has no proof of length at most k; $\vdash_k \neg\varphi$ says that its negation has such a proof; and $\vdash \neg\Box_k\varphi$ says that S proves that no such proof of φ exists. By soundness, the second and the third implies the first. The first does not, in general, imply the second, as failure to find a proof of φ does not amount to finding a proof of its negation. For fixed k and φ, a failed search can itself be proven in S, but the proof need not have length at most k. This distinction between a search failing and its failure being provable within the same budget is the subject of Section 4.2.

3.2 The mathematics behind the outcomes

With the formal system in place, programs can reason about their opponent's behavior using the formalism defined by S. The system alone, however, does not tell us much about the actual reasoning, only it's language. We

[12] Including the transpose as an axiom ensures that transposition preserves proof length without requiring additional derivation steps. As defined in Definition 3.2, the transpose is obtained by simultaneously exchanging c_C and c_D.

[13] Equivalently, of any model $\mathcal{M}$ of T_{Act}.

Table 1. Notation for bounded provability and truth. The sentence $\Box_k\varphi$ belongs to the language of S; the assertions $\vdash_k \varphi$ and $\models \varphi$ are made in the metatheory.

Notation	Meaning
$\Box_k\varphi$	A sentence expressing provability in S within budget k
$\vdash_k \varphi$	A proof of φ in S of length at most k exists
$\models \varphi$	φ is true in $\mathbb{N}$ under the standard reading

now present some of the mathematical tools underlying the proofs; partly to show how reasoning in S actually proceeds, and partly because the results themselves are worth seeing. The same tools also yield results beyond those of Critch et al. (2022). We settle several of their open problems and refute one of their examples. In what follows, we state the general ideas and leave the precise definitions, outcomes and proofs in Appendix B.

A fundamental property in our system S is that proved statements are true, i.e that S is sound (Proposition B.1). Soundness yields both positive and negative results: positively, it shows that `DupocBot` is unexploitable (Proposition B.3; see also the discussion in Example 2.6); negatively, since S proves only true statements, a false hypothesis has no proof of any length, so any search for one fails (e.g. Proposition B.2).

Complementary to soundness, we have completeness: if a statement is true for any model of S, then S should be able to prove it (Theorem B.4 and Corollary B.5). This gives us for instance that `DupocBot` and `CooperateBot` mutually cooperate (Proposition B.6).

Soundness and completeness, however, both presuppose that we can judge whether a hypothesis is true independently of the search for its proof. This fails when programs reason about each other, e.g., when `DupocBot`(k) faces itself, its hypothesis is true only if its own search succeeds (Example 2.6). Such self-fulfilling hypotheses are resolved by Löb's theorem (Löb 1955), a core theorem (Critch 2019) in mathematical logic at the heart of Critch et al. (2022)'s results. We state it as Theorem B.7 and re-derive the surprising result of `DupocBot`'s self cooperation (Theorem B.8). Beyond self-cooperation, Löb's theorem resolves several of Critch et al.'s open problems and refutes their Example 5.3(a) (Appendix B.3).

3.3 Transposition and `DupocBot` vs. `CupodBot`

Aside the well-known tools from above, we introduce a new technique, which makes use of the inherent syntactic symmetry between the two actions C and D in programs playing the Prisoner's dilemma. The *transposition* between the two actions allows us to prove a standing open-problem in the field, the outcome of `DupocBot` vs. `CupodBot`[14]. For all proofs and additional details for this section, refer to Appendix B.2.

The two action constants play symmetric roles in S, suggesting that exchanging them should transform any proof into another (transposed) proof of the same length. Note that the programs considered here make no reference to payoffs, the semantic content of a game, so this symmetry concerns the syntax of the programs and the proof system. We make this idea precise in the following definition.

Definition 3.2 (Transposition). Let φ be a formula of S. We define the transpose $\mathcal{T}\varphi$ to be the formula obtained by replacing every occurrence of c_C by c_D and, simultaneously, every c_D by c_C.

Since $\mathcal{T}$ touches only constants, it commutes with every logical symbol,

$$\mathcal{T}(\neg\varphi) = \neg\,\mathcal{T}\varphi, \qquad \mathcal{T}(\varphi \to \psi) = \mathcal{T}\varphi \to \mathcal{T}\psi, \qquad \mathcal{T}(\forall x\,\varphi) = \forall x\,\mathcal{T}\varphi,$$

and it is an involution $\mathcal{T}\mathcal{T}\varphi = \varphi$. With this definition in mind, we have the following result.

[14] It is also Critch et al. (2022)'s Open Problem 3.

Theorem 3.3. *For every sentence φ of S and every budget $k \in \mathbb{N}$,*

$$\vdash_k \varphi \quad \Longleftrightarrow \quad \vdash_k \mathcal{T}\varphi.$$

Naturally, we want to extend transposition to the program's source code itself.

Definition 3.4 (Transposed program). For a program π, the transposed program $\mathcal{T}\pi$ is obtained by exchanging the actions C and D everywhere in the source of π, including inside the hypotheses it searches for.

The transpose of `DupocBot`(k) is "Cooperate Unless Proof Of Defection", $\mathcal{T}$`DupocBot` = `CupodBot`, given below (using the notation of Section 2.1):

```
def CupodBot(k)(π):
    if proof_search(k, π, run(π‖CupodBot(k)) = D):
        return D
    else:
        return C
```

With both definitions above, we can arrive at the first symmetry result we have for our programs. For programs π, π' and an action a, write $\mathrm{Plays}(\pi, \pi', a)$ for the sentence of S stating that π, run on the source of π', plays a. The two transpositions agree on such sentences.

Theorem 3.5. *For all programs π, π' and every action a,*

$$\mathcal{T}\,\mathrm{Plays}(\pi, \pi', a) \;=\; \mathrm{Plays}(\mathcal{T}\pi, \mathcal{T}\pi', \mathcal{T}a).$$

where $\mathcal{T}a$ is the other action.

We now apply Theorem 3.3 and Theorem 3.5 directly to `DupocBot` and `CupodBot`, obtaining the outcome conjectured in Open Problem 3 of Critch et al. (2022) for our proof system S.

Theorem 3.6 (`DupocBot` against `CupodBot`). *For every $k \in \mathbb{N}$,*

$$\mathrm{outcome}\big(\texttt{DupocBot}(k), \texttt{CupocBot}(k)\big) = (D, C).$$

Let us give here a sketch of the proof. The two bots search for transposed hypotheses with the same proof-length budget. By Theorem 3.3 and 3.5 their proof searches therefore succeed or fail together. If both succeed, `DupocBot` cooperates and `CupocBot` defects. But success of `DupocBot`'s search means it has proved that `CupocBot` cooperates, contradicting soundness. Thus both searches fail, and the bots take their default actions, i.e., `DupocBot` defects and `CupocBot` cooperates. The full proof is given in the appendix.

4 A Lean Formalization

The previous section shows that finding the outcomes of open-source programs involves Löbian, self-referential arguments that are laborious and error-prone by hand, and the complexity of these proofs grows with the complexity of the programs involved and of the game played. To make this analysis more systematic and robust, we formalize the theory in a formal verification language, both to ensure outcomes are machine-checked, and to facilitate the search for new proofs using automated tools, such as large language models. For these reasons, we choose Lean 4 (Moura and Ullrich 2021), a proof assistant language that has gained traction in the literature for its ability to sustain large-scale formalizations of modern mathematics, such as the community-owned mathlib library (The mathlib Community 2020) and the ongoing Lean proof of Fermat's Last Theorem (Buzzard and Taylor 2025), and for its role as the de facto target language for learning-based theorem proving, from the miniF2F benchmark and the LeanDojo toolchain (Yang et al. 2023; Zheng et al. 2022) to the current generation of neural provers (Hubert et al. 2026; Lin et al. 2026; Ren et al. 2025; Wang et al. 2025). Lean is also home to the closest

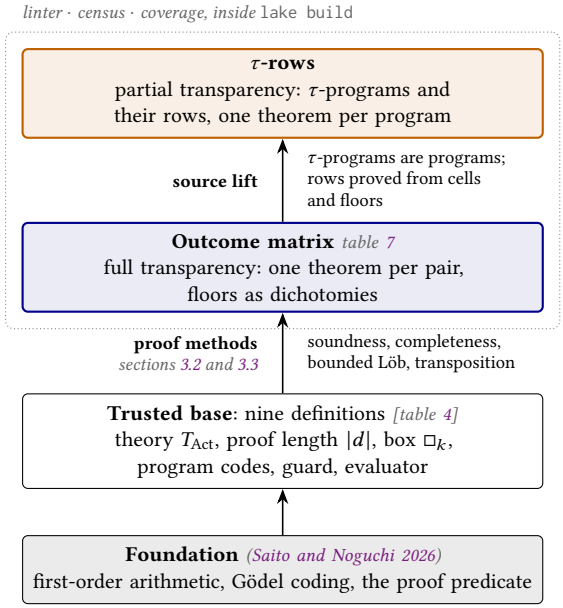


Fig. 1. Architecture of the formalization. Everything shown is checked by the Lean kernel and rests on no project axioms. The Foundation library is trusted as a library, nine definitions form the trusted base, and every outcome cell (blue, full transparency) and every τ-row (orange, partial transparency) is a theorem or an explicitly stated open case, validated by a linter, a census and a coverage check inside `lake build`.

prior work: `ModalAgents` (Berns 2026) mechanizes the modal agents of Bárász et al. (2014), but with unbounded provability, so it has no proof lengths, no evaluator and no transparency. Ours is, to our knowledge, the first mechanization of resource-bounded open-source game theory.

Our formalization rests on four commitments, to make it robust against possible silent failures and inconsistencies. First, we do not introduce any axioms, apart from Lean's three standard axioms (`propext`, `Quot.sound`, `Classical.choice`), so no project-specific assumptions are made. Second, all our theorems are `sorry`-free, ensuring that every statement is entirely proved. Third, the outcomes the rest of the paper consumes follow fixed, typed templates, enforced by the build in both directions: a linter checks that every tagged theorem is a literal instance of its template whose programs and outcome match its name, and a census checks the converse, that every outcome proved in the library is tagged. Every pair of programs used in our experiments is thereby either a proved cell or an explicitly stated open case (Section 4.2). Finally, we build upon the Foundation library (Saito and Noguchi 2026), which is to the best of our knowledge the most advanced library in mathematical logic. On top of their work, we add the nine definitions of Table 4, that constitute our base. The result, overviewed in Figure 1, is a kernel-checked Lean 4 formalization[15] of open-source game theory under full and partial transparency.

The remainder of this section is organized as follows. Section 4.1 walks through the three components of the mechanization: programs, outcomes and the system S itself, built on Foundation. Section 4.2 then turns to the open cells and traces them to a single open problem of bounded proof complexity: whether T can show, within budget k, that a given false sentence has no proof of length at most k.

[15]The code is available at https://github.com/ColombanD/open-source-game-theory/tree/osgt-formalization-transparency-cooperation (Duclaux, Formenti, Cobben, T. J. Zhang, et al. 2026).

4.1 Details of the Mechanization

Open-source game theory rests upon three main components that we expose here: programs, outcomes, and the formal system S. Definitions, statements and architecture were designed by the authors, but most of the Lean source was written by LLM agents built on Anthropic's Claude Opus and Fable model families (Anthropic 2026a,b), run through Claude Code on a Claude subscription account. The formalization comprises about 160,000 lines of Lean, written in an estimated 85 hours of agent activity; Appendix C.2 breaks this down by component. Nothing an agent writes is trusted: every theorem is checked by the Lean kernel, and the axiom census, the linter and the coverage checks run inside the build. The provenance of a proof is therefore irrelevant to its validity, which is the point of mechanizing.

4.1.1 Programs. A program in our mechanization is a term of the inductive type `Prog`[16], and the set Σ of admissible programs from Section 2.1 is realized as a finite collection of such terms. Two `Prog` constructors are important here: `.search` for logical reasoning, and `.sim` for simulation. Below, we present two examples: our proof-based running example, Dupoc, and a simulation-based program, DBot.

```
def DupocBot (k : Nat) : Prog :=
  .search k                          -- the proof search constructor
    (.plays .opp .self Action.C)     -- hypothesis: opponent plays C against me
    (.const Action.C)                -- proof found: cooperate
    (.const Action.D)                -- otherwise: defect
```

```
def DBot : Prog :=
  .ite                               -- if-then-else
    (.sim .opp (.bot DefectBot))     -- simulate opponent against DefectBot
    Action.C                         -- test action C
    (.const Action.D)                -- if opponent cooperates, defect
    (.const Action.C)                -- else, cooperate
```

The hypothesis a `.search` looks for is a value of a second inductive type, `Formula`, which has constructors stating atomic plays (`.plays p q a`, "p plays a against q", where `.self` and `.opp` stand for the two players of the current match); implications and negations of such statements; a syntactic equality `.eq p q`, "p is literally the program q"; and the bounded provability operator $\Box_k$ of Section 3.1: `.box k` ψ, "ψ has a proof of length at most k". The three remaining constructors implement partial transparency: `.tvote` runs the threshold vote of `TauDupocBot`(k, α) over a list of weighted hypotheses, and `.sys` with `.selfIdx` bind the mutually recursive system that two voting programs probing each other require.

4.1.2 Outcomes. As stated in the introduction to this section, outcome theorems (i.e. the per-pair matchups between programs in the library) follow a strict template to ensure consistency: an `outcomeA` attribute, and a fixed statement syntax (see example below). Programs are run by an evaluator `EvalGraph n me opp p a`, read "with fuel n, in the match of `me` against `opp`, the program `p` plays `a`". Nested simulation makes termination non-structural, hence the explicit fuel: a program plays an action if some amount of fuel returns it, and that action is unique because the evaluator is deterministic and monotone in the fuel. Below is one of the running example's cells, stated on the template, whose first argument fixes the budget regime (`.universal`: at every budget k; `.eventual`: at every sufficiently large k).[17]

```
@[outcomeA]
```

[16] `Prog` has ten constructors. The seven base constructors all appear in the two examples below; the remaining three extend the language to partial transparency.

[17] A careful reader will recognize this example as being the outcome certificate of our headline Dupoc vs Cupod proof.

```
theorem outcome_DupocBot_vs_CupodBot :
    OutcomeSpecA .universal (bot DupocBot) (bot CupodBot) (some (.D, .C))
-- unfolds to: ∀ k, DupocBot k plays D against CupodBot k,
--             and CupodBot k plays C against DupocBot k
```

Under partial transparency the template is a row vector of all outcomes between a fixed bot and the zoo. The evaluator's `.search` clause asks whether a proof of length at most k exists, in the formal system S, defined in Section 3.1 and constructed below. This is a bounded search over proof codes, hence decidable in principle but far from feasible: the content of an outcome is carried by its theorem, not by running the programs.

4.1.3 The formal system S. Perhaps the most interesting mechanization result is the actual formalization of S, a language of arithmetic with two action constants, Peano Arithmetic with the action axiom, and proofs measured by their number of symbols. In `lean`, we build S on top of the Foundation library. We sketch the mechanization here and leave further details in Appendix C.

S is a theory of arithmetic, so its sentences speak only about natural numbers, since its signature contains only 0, 1, $+$, $\cdot$, $<$ and the two action constants. For S to reason about sentences, proofs and programs, each of these objects is assigned a natural number, its Gödel code. A statement about proofs or programs then becomes a statement about numbers, which S can express.

Furthermore, formulas have no truth value on their own, only in a logical structure. For us, that structure is $\mathbb{N}$ because codes of proofs are actual natural numbers by construction. This lets "some number codes a proof" and "some proof exists" say the same thing:

$$\vdash_k \varphi \quad \Longleftrightarrow \quad \mathbb{N} \models \Box_k \varphi \quad \Longleftrightarrow \quad \text{some derivation of } \varphi \text{ in } T_{\mathrm{Act}} \text{ has at most } k \text{ symbols},$$

Proof length. Finally, modeling the length of a proof requires careful accounting. In the Foundation library, proofs are derivations in a sequent calculus[18]. We define the length $|d|$ of a derivation d as its number of symbols, counted over every node of the tree; the precise count is given in Appendix C.

4.2 What S Does Not Decide

The tools of Section 3.2 show us how to settle certain types of hypotheses of our `proof_search` function. However, one kind of hypothesis escapes them: one that is true, but true only because some other search fails. Whether such a hypothesis is found is a question the tools cannot answer, and one that S itself leaves open.

A search of budget k fails when no proof of length at most k exists. One might expect this failure to be an easy fact, provable within the same budget k. To see why it is not, suppose any failed k-search could be certified within budget k: for any hypothesis φ with else-action b, $\nvdash_k \varphi$ implies $\vdash_k$ (me plays b). Then we obtain a contradiction with the following anti-diagonal program[19]:

```
-- "If I can prove that I defect against myself, cooperate"
def AntiBot (k : Nat) : Prog :=
  .search k
    (.plays .self .self Action.D)      -- hypothesis: I play D against myself
    (.const Action.C)                  -- proof found: cooperate
    (.const Action.D)                  -- otherwise: defect
```

[18] A derivation is a tree in which every node carries a *sequent*, a finite set of formulas read as a disjunction, and every edge is an inference rule. Foundation's calculus is one-sided and has the cut rule.

[19] Indeed, set φ = (`AntiBot` plays D vs `AntiBot`) and suppose, for contradiction, that $\vdash_k \varphi$. Then soundness (Proposition B.1) implies that `AntiBot` defects. However, a provable hypothesis is a found hypothesis, which makes `AntiBot` take its then branch and play C. Impossible, because a program plays only one action. Hence $\nvdash_k \varphi$, so `AntiBot` takes its *else* branch and plays D: the sentence φ is true. But by the hypothesis, $\nvdash_k \varphi$ would imply $\vdash_k \varphi$, a contradiction.

A certificate that a search failed may therefore cost more than the search itself, and nothing in S says in general how much more. This matters as soon as one program's hypothesis depends on another program's *else* play. Consider `CupodTrollBot`, which cooperates with any program except if its opponent is literally `CupodBot`:

```
def CupodTrollBot (k : Nat) : Prog :=
  .search k
    (.eq .opp (CupodBot k)) -- test if opponent is literally CupodBot
    (.const Action.D)       -- then, defect
    (.const Action.C)       -- else, cooperate
```

Against `DupocBot`, its hypothesis is false, so by soundness its search fails and it cooperates, at every budget. `DupocBot`'s hypothesis, "`CupodTrollBot` plays C against me", is therefore true. But it is true only because `CupodTrollBot`'s search failed, and to find it `DupocBot` needs, within its budget k, a proof that a search (of the same budget k) fails: exactly the kind of certificate we just showed is not free. None of our tools in the previous section decides whether the certificate above fits, which turns out to rest on a question about consistency.

Write $\mathrm{Con}_S(n)$ for the sentence of S stating "no proof of length at most n ends in a contradiction". For each fixed n, S proves $\mathrm{Con}_S(n)$[20], but we do not know how long the proof must be. It cannot be shorter than n^ϵ for some $\epsilon > 0$ (Pudlák 2017, Thm. 3.1), a bound due to Friedman (unpublished, 1979) and Pudlák (1986). Our open cells rest on whether a proof of length about n exists, which, to our knowledge, remains an open question. We call this the bounded consistency problem.

Definition 4.1 (Floor, hinge, sides). A cell is a floor if its outcome depends on whether one of its searches finds a true hypothesis φ_k, and showing that it does would settle the bounded consistency problem: S would then prove $\mathrm{Con}_S(n)$ within about n symbols[21]. Precisely, writing $H_k := (\vdash_k \varphi_k)$, there are two outcomes $o^- \neq o^+$ such that, for all large k,

$$\neg H_k \implies \text{outcome } o^- \qquad \text{and} \qquad H_k \implies \text{outcome } o^+.$$

The statement H_k is the hinge of the floor. The outcome o^- is its negative side, on which the hypothesis is true but not found within budget, and o^+ is its positive side, on which it is found.

The cell of `DupocBot` against `CupodTrollBot` is a floor. Its hinge H_k states that `DupocBot`'s hypothesis has a proof of length at most k, its negative side is (D, C) and its positive side is (C, C).

The consistency problem above is not on artefact of budgets, as it already appears, unbounded, in Bárász et al. (2014), where `PrudentBot` must establish that its opponent defects against `DefectBot`. For an opponent that searches for proofs, this means that a search in PA fails, which PA cannot certify without its own consistency, so the check runs in the stronger system PA+1. With budgets there is no stronger system to move to, only a larger budget. A self-fulfilling fact is certified within the budget at which it holds, whereas the failure of a search of budget k needs a proof of Con_S at about k, and whether that proof fits within k is the bounded consistency problem.

Since that problem is open, we fix a convention. Unless stated otherwise we report every floor on its negative side, the one on which no failure certificate fits its budget. This is a labelled convention, not a theorem, and Section 5 repeats the analysis with every floor on its positive side. The two sides give different outcomes, so no theorem of ours assumes either.

[20] Since S is sound, $\mathrm{Con}_S(n)$ is true for every n, and S proves it for each fixed n, because it is then a finite check

[21] Precisely, if H_k is the hinge of such a floor then, for all large k, $H_k \implies \vdash_{k+O(\log^2 k)} \mathrm{Con}_S(k - O(\log k))$: if the hypothesis has a proof of length at most k, then S proves within about k symbols that it has no contradiction of length about k. The length is k only up to a small term because the proof of Con_S is the proof of the hypothesis followed by a few more lines, and these lines must write out the number k.

5 Transparency, Cooperation, and Equilibrium Selection

We study how transparency affects both the cooperation sustainable in program equilibrium and the cooperation selected by replicator dynamics, first under full transparency and then under partial transparency. We fix a finite library of programs and build the meta-game payoff matrix from their pairwise outcomes (Table 2). Section 5.1 treats full transparency. We enumerate the program equilibria and study equilibrium selection under replicator dynamics. Section 5.2 then varies the transparency map and measures how the cooperation sustainable in equilibrium, and the cooperation selected by the dynamics, change with the level of transparency.[22]

Example 5.1 (Prisoner's Dilemma, cont'd). Throughout this section, we instantiate Example 2.1 with $T = 3$, $R = 2$, $P = 0$, and $S = -1$:

	C	D
C	$(2, 2)$	$(-1, 3)$
D	$(3, -1)$	$(0, 0)$

$$3 > 2 > 0 > -1.$$

Mutual cooperation thus gives each player 2, whereas mutual defection, the unique Nash equilibrium of G_{PD}, gives each player 0.

5.1 Full Transparency

5.1.1 Program Library. While the results that follow do depend on the choice of the program library Σ, we have taken care to consider several different libraries. Our focus here is on optimal rather than average behavior, so adding dominated bots to a library would leave our results unchanged. Moreover, programs such as `DupocBot` are provably unexploitable (proposition B.3), which makes it difficult to find programs that outperform them. We start with the base case of full transparency, i.e. of open source game theory; fix a base library $\Sigma = \{\pi_1, \ldots, \pi_N\}$ containing $N = 8$ bots

$$\Sigma = \{\texttt{CooperateBot}, \texttt{CupodBot}, \texttt{DBot}, \texttt{DefectBot}, \texttt{DupocBot}, \texttt{EBot}, \texttt{OBot}, \texttt{TitForTatBot}\},$$

which is defined in Figure 2. All formal program source codes can be found in Appendix G.

We provide proofs of all outcomes for all pairs of programs in our Lean library (Section 4). These proofs were produced either by hand or by an LLM proof agent that drives the Lean kernel. The agent is built on Anthropic's Claude Opus and Fable model families (Anthropic 2026a,b) (Appendix C.2). Table 2 summarizes the outcomes.

5.1.2 Enumerating Program Equilibria. We are interested in understanding which programs are optimal to play in equilibrium. To do so, we enumerate all program equilibria of the meta-game G^{prog} following the approach described in Appendix D.1. For the eight-program library Σ at full transparency, we find 21 extreme equilibria across three connected components (Table 8).

The three components differ in the probability of mutual cooperation. In the first, every equilibrium yields (C, C) with probability one and gives each player the cooperative payoff $R = 2$. In the second, there is a single symmetric equilibrium in which each player mixes over `CooperateBot`, `OBot`, and `EBot`. Mutual cooperation occurs with probability $11/18$, and each player's expected payoff is $3/2$. In the third, every equilibrium yields (D, D) with probability one and gives each player a payoff of zero. Thus the unique equilibrium outcome of the underlying Prisoner's Dilemma persists, but the game also sustains cooperative and partially cooperative equilibria, which are not present in the base Prisoner's dilemma. The role of `DupocBot` is clearest in self-play. In the underlying game, a player facing cooperation can gain by defecting. In the meta-game, switching programs can also change the opponent's action. In particular, `DupocBot` cooperates with itself, but a player who switches to a program that defects against it faces defection in return. This makes mutual cooperation a best response

[22] Code for all experiments is available at https://github.com/ColombanD/open-source-game-theory/tree/osgt-formalization-transparency-cooperation and https://github.com/RiccardoFormenti/partial-transparency-game-theory.

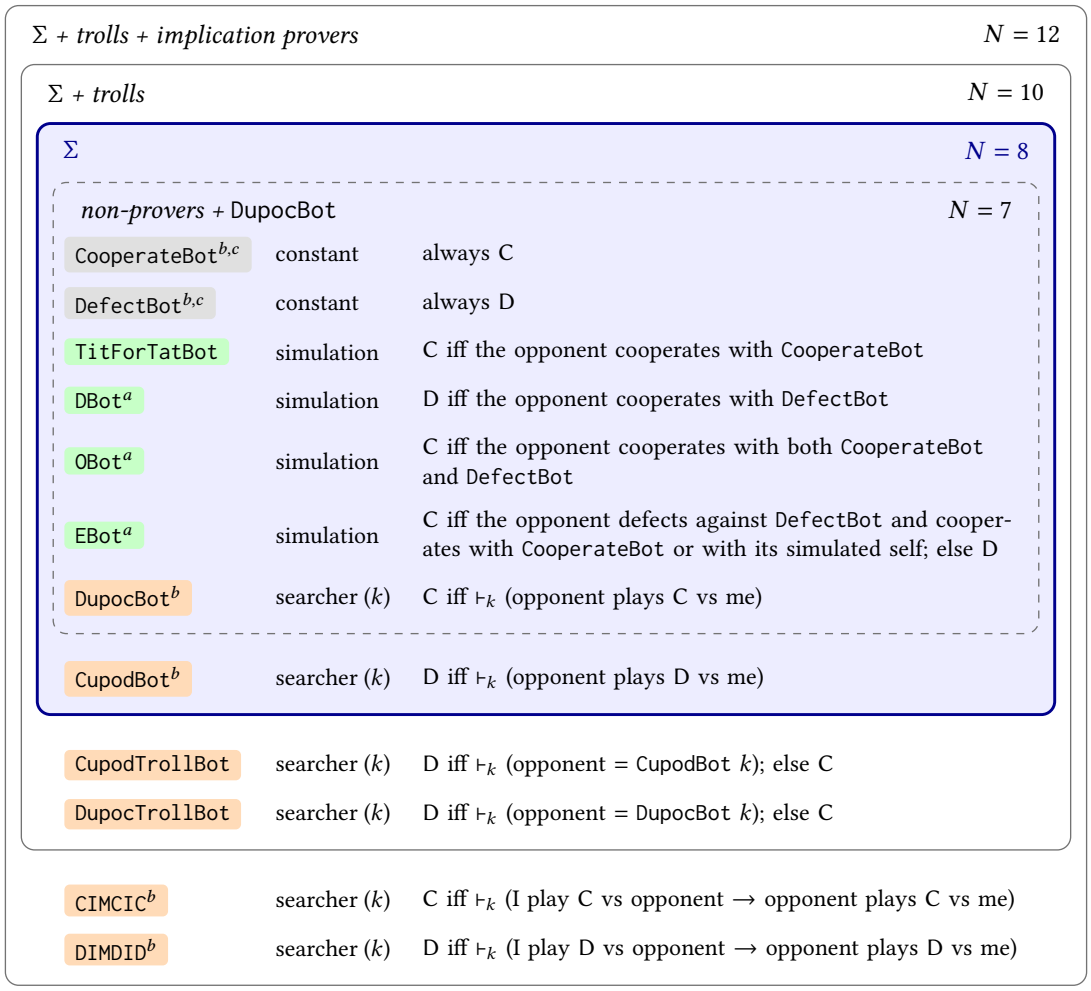


Fig. 2. The bots used in this paper, as four nested libraries. The Σ library (filled) is the one analysed throughout; the two outer zoos are the robustness extensions. Provenance: [a] transcribed from the LessWrong source-visible tournament (Mennen 2013); [b] Critch et al. (2022); [c] Bárász et al. (2014), bounded here; untagged bots are ours. Further bots, not analysed here, are in the repository (Duclaux, Formenti, Cobben, T. J. Zhang, et al. 2026).

for both players in DupocBot self-play. The enumeration shows that its role extends beyond self-play in that in every fully cooperative equilibrium of this library, both players assign probability at least 1/9 to DupocBot. No other program appears in every such equilibrium.

5.1.3 Equilibrium Selection under Replicator Dynamics. Which of these equilibria does a population reach when programs that earn higher payoffs become more common? We study this question using replicator dynamics in G^{prog}. Programs are matched at random, and a program's share grows whenever its expected payoff exceeds the population mean. For the eight-program library Σ at full transparency, every trajectory we simulated from a uniformly drawn initial population converged to the fully cooperative component (Appendix E.2). Thus,

Table 2. The symmetric outcome matrix of the eight-bot library Σ in S, at a common proof budget k; a cell gives the row bot's certified action first. Cells hold for every sufficiently large k. Shaded cells represent mutual cooperation (green) and mutual defection (grey). Orange cells‡ rest on the hinge of Definition 4.1, where we show its negative side. Highlighted names (yellow) are empirically unexploitable bots (see Proposition B.3 for more on this)

	CooperateBot	DefectBot	TitForTatBot	DBot	OBot	EBot	DupocBot	CupodBot
CooperateBot	CC	CD	CC	CD	CC	CD	CC	CC
DefectBot	DC	DD	DD	DC	DD	DD	DD	DD
TitForTatBot	CC	DD	CC	DC	CD	DC	CC	CC
DBot	DC	CD	CD	DD	CD	CD	CD‡	CC
OBot	CC	DD	DC	DC	DD	DC	DD	DD
EBot	DC	DD	CD	DC	CD	CC	CD‡	CC
DupocBot	CC	DD	CC	DC‡	DD	DC‡	CC	DC
CupodBot	CC	DD	CC	CC	DD	CC	CD	DD

although mutual defection remains an equilibrium outcome, in every run the dynamics converge to a cooperative equilibrium. The initial composition influenced which programs persisted in the long run. In some populations only DupocBot was played at equilibrium, while in most it coexisted with both CooperateBot and TitForTatBot. Every pairing among these three programs yields mutual cooperation, so each earns $R = 2$ regardless of its population share. Thus, under full transparency, all sampled populations reached the same cooperative outcome through different mixtures of programs. A positive share of population of DupocBot is required for cooperation in equilibrium.

The above findings replicated across every program library shown in Figure 2 (see Appendix E.3).

5.2 Partial Transparency

Under full transparency, switching programs can change the opponent's response and thus eliminate the gain from defection (Section 5.1). We perform the comparative statics exercise of holding payoffs and the program library fixed and varying what each program knows about its opponent's program; we follow throughout the setup of Section 2.2.1. In particular, we show that at every caution level tested, both the maximum cooperation sustainable in equilibrium and the cooperation selected in equilibrium by replicator dynamics are increasing in transparency.

5.2.1 Experimental Setup. We use the eight-program library Σ of Section 5.1.1; for precise programs definitions, see Appendix G and H. Under the threshold rule of Section 2.2.1, a program cooperates if its belief assigns probability at least α to opponents against which itself would cooperate, for the caution parameter $\alpha \in [0, 1]$. A more cautious program thus requires stronger evidence before it plays C. We use two families of transparency maps. Behavioral maps measure the distance between two programs by counting the opponents against which their actions differ. Syntactic maps use the normalized edit distance between the syntax trees of their source code; see Appendix F.1 for more details. To compare the families, we measure the fraction of uncertainty about the opponent's identity that each map τ removes, denote by $t(\tau)$ and called the transparency level. At $t = 0$, the signal is independent of the opponent's identity, i.e., players acts as if they were in the base game. At $t = 1$, it identifies the opponent exactly, the setting of full transparency in open source game theory. Appendix F.1 gives the constructions.

In each family, we sweep across transparency-caution pairs, transparency $t \in \{0, 0.01, 0.02, \ldots, 0.99, 1\}$, and caution $\alpha \in \{0, 0.2, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1\}$. At each grid point we enumerate the program equilibria, the Nash equilibria of the induced meta-game G^{τ}_{PD} (Appendix D.1). To study equilibrium selection, we compute replicator dynamics from populations drawn uniformly from the simplex (Appendix D.2). In particular, we measure $\Pr[(C, C)]$, the probability of mutual cooperation.

5.2.2 Equilibrium Cooperation and Selection. We observe that in the partially transparent Prisoner's Dilemma, greater transparency increases both the cooperation that players can sustain in equilibrium and the cooperation that payoff-based equilibrium selection reaches. At each caution level, a fully cooperative program equilibrium first appears at a critical level of transparency and remains available at every higher level. Cooperation can then be self-enforcing because switching to an exploitative program can induce the opponent to defect in return. The same threshold holds in selection in both signal families. Every simulated population converges to mutual defection below it and to a fully cooperative equilibrium at or above it. In our motivating oversight example, that cooperative equilibrium is collusion.

Figure 3 shows this selection in the behavioral family. Mutual defection remains an equilibrium at every point shown, yet all initial populations attain the maximum equilibrium cooperation at each (t, α).

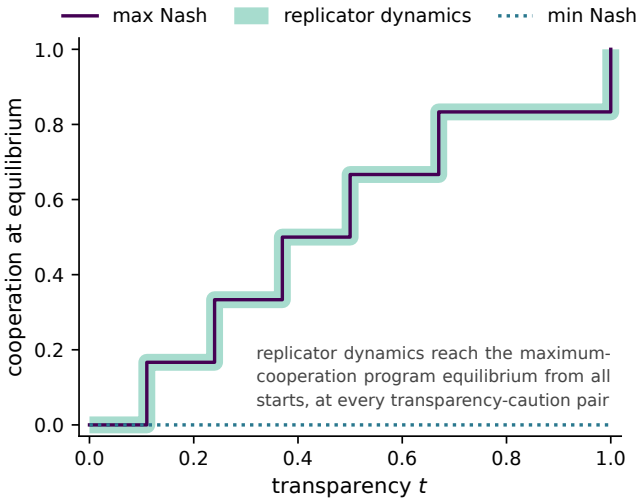


Fig. 3. Equilibrium cooperation and selection under behavioral transparency. The solid and dotted lines show the maximum and minimum cooperation $\Pr[(C, C)]$ at equilibrium, averaged over the six caution levels $\{0.5, 0.6, 0.7, 0.8, 0.9, 1\}$ at each t. The green trace shows the outcome of replicator dynamics and coincides with the maximum throughout.

Cooperation exhibits a sharp threshold in transparency needed to enable it (Figure 4(a)). Below the threshold, the only equilibrium outcome is defection, and thus every simulated population converges to it. As transparency reaches a critical threshold, full cooperation becomes an equilibrium outcome, and replicator dynamics select it in every run at or above that level. At $\alpha = 0.8$, every equilibrium yields mutual defection at $t = 0.49$. At $t = 0.50$, a fully cooperative equilibrium appears and every simulated population reaches it. Each rise of $1/6$ in Figure 3 thus marks one more caution level crossing its threshold.

Greater caution requires more transparency for full cooperation over the six caution levels shown in Figure 4(b). Let t^* be the lowest sampled transparency at which a fully cooperative equilibrium exists. At $\alpha = 0.5$, t^* is 0.11

for behavioral signals and 0.07 for syntactic signals. The threshold rises at each subsequent caution level. At $\alpha = 1$, when programs require certainty before cooperating, both families require full transparency.

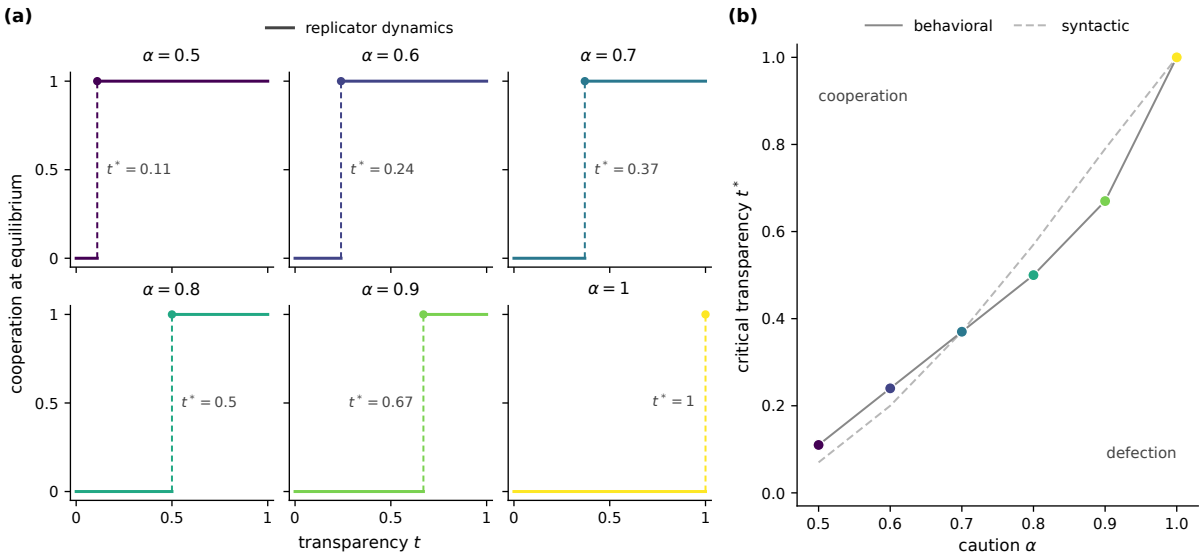


Fig. 4. Critical transparency for full cooperation. **(a)** At each caution level in the behavioral family, selected cooperation jumps from zero to one at t^*. **(b)** The solid and dashed lines connect the sampled thresholds for behavioral and syntactic signals, respectively. In both families, every simulated population defects below t^* and cooperates fully at and above it. The transparency grid has spacing 0.01. Across both panels, same colors denote same caution levels.

AI Use Statement

In this work, we used generative AI tools to provide feedback on research methodology and experimental design, implement methods, cleaning and formatting the datasets, and check mathematical arguments. We have use AI agentic tools to assist in the Lean4 formalization (Appendix C.2). We have not used AI tools to propose or refine hypotheses, to interpret experimental results, or to set the direction of the research. Additionally, we used generative AI tools to identify, summarize, and analyze relevant literature, write and refine software code, create or modify scientific figures, and polish and contribute to the manuscript's writing and editing. The models used include GPT-5.6 Sol, GPT-6 Astra (OpenAI) and Fable 5, Fable 5.1, Opus 5, Opus 5.5 (Anthropic). We, the authors, take full responsibility for the final contents of this work, including text, claims or artifacts produced with the aid of generative AI tools.

Acknowledgments

This work was supported by Renaissance Philanthropy through The Alignment Project, a funding programme of the UK AI Security Institute (AISI), under the project "Formal Verification of Collusion in Transparent AI Agents".

References

Anthropic. Sept. 2026a. *System Card: Claude Fable 5.1 & Claude Mythos 5.1.* Tech. rep. Published September 1, 2026. Accessed 2026-09-18. Anthropic, (Sept. 2026).

Anthropic. May 2026b. *System Card: Claude Opus 4.8.* Tech. rep. Published May 28, 2026. Accessed 2026-09-18. Anthropic, (May 2026).

D. Avis, G. D. Rosenberg, R. Savani, and B. Von Stengel. 2010. "Enumeration of Nash equilibria for two-player games." *Economic theory*, 42, 1, 9–37.

M. Bárász, P. F. Christiano, B. Fallenstein, M. Herreshoff, P. LaVictoire, and E. Yudkowsky. 2014. "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic." *CoRR*, abs/1401.5577. arXiv: 1401.5577.

P. Battigalli. 1987. "Comportamento razionale ed equilibrio nei giochi e nelle situazioni sociali." *typescript, Bocconi University*.

P. Battigalli, S. Cerreia-Vioglio, F. Maccheroni, and M. Marinacci. 2016. "Analysis of information feedback and selfconfirming equilibrium." *Journal of Mathematical Economics*, 66, 40–51.

D. Bergemann and S. Morris. 2019. "Information design: A unified perspective." *Journal of economic literature*, 57, 1, 44–95.

A. M. Berns. 2026. *Formalized Agent Foundations*. Lean 4 library, Apache-2.0. Git commit `58e9231`, accessed September 7, 2026. (2026). https://github.com/A-M-Berns/Formalized-Agent-Foundations.

K. Buzzard and R. Taylor. 2025. *FLT: An Ongoing Lean Formalization of Fermat's Last Theorem*. https://github.com/ImperialCollegeLondon/FLT. Imperial College London. Blueprint: https://imperialcollegelondon.github.io/FLT/blueprint/. (2025).

E. Cooper, C. Oesterheld, and V. Conitzer. 2025. "Characterising Simulation-Based Program Equilibria." In: *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*. Ed. by T. Walsh, J. Shah, and Z. Kolter. AAAI Press, 13735–13744. doi:10.1609/AAAI.V39I13.33501.

A. Critch. 2019. "A parametric, Resource-Bounded Generalization of löB's Theorem, and a robust Cooperation criterion for Open-Source Game Theory." *J. Symb. Log.*, 84, 4, 1368–1381. doi:10.1017/JSL.2017.42.

A. Critch, M. Dennis, and S. Russell. 2022. "Cooperative and uncooperative institution designs: Surprises and problems in open-source game theory." *CoRR*, abs/2208.07006. arXiv: 2208.07006. doi:10.48550/ARXIV.2208.07006.

C. Duclaux, R. Formenti, P. Cobben, B. Schölkopf, and Z. Jin. 2026. "Proving Your Way to Cooperation: Formalizing Proof-Based Open Source Game Theory in Lean." In: *3rd AI for Math Workshop: Toward Self-Evolving Scientific Agents*.

C. Duclaux, R. Formenti, P. Cobben, T. J. Zhang, B. Schölkopf, and Z. Jin. 2026. *Open-Source Game Theory in Lean 4*. https://github.com/ColombanD/open-source-game-theory/tree/osgt-formalization-transparency-cooperation. Lean 4 library and analysis code. Release `osgt-formalization-transparency-cooperation`, Git commit `50097cb`, accessed 2026-11-06. (2026).

H. B. Enderton. 2001. *A Mathematical Introduction to Logic*. (2nd ed.). Academic Press.

R. Formenti. 2026. "Self-Confirming Equilibrium under Model Misspecification." *arXiv*, arXiv.2610.35692. https://arxiv.org/abs/2610.35692 eprint: 2610.35692.

D. Fudenberg and D. K. Levine. 1993. "Self-confirming equilibrium." *Econometrica: Journal of the Econometric Society*, 523–545.

I. Gilboa and M. Marinacci. 2016. "Ambiguity and the Bayesian paradigm." In: *Readings in formal epistemology: Sourcebook*. Springer, 385–439.

K. Gödel. 1930. "Die Vollständigkeit der Axiome des logischen Funktionenkalküls." *Monatshefte für Mathematik und Physik*, 37, 1, 349–360. doi:10.1007/BF01696781.

R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger. 2024. "AI Control: Improving Safety Despite Intentional Subversion." In: *Forty-first International Conference on Machine Learning, ICML* (Proceedings of Machine Learning Research). Vol. 235.

L. Hammond et al.. Feb. 2025. *Multi-Agent Risks from Advanced AI*. Technical Report 1. Cooperative AI Foundation, (Feb. 2025). arXiv: 2502.14143 [cs.MA]. doi:10.48550/arXiv.2502.14143.

J. Hofbauer and K. Sigmund. 2003. "Evolutionary game dynamics." *Bulletin of the American mathematical society*, 40, 4, 479–519.

B. Holmström. 1979. "Moral hazard and observability." *The Bell journal of economics*, 74–91.

T. Hubert et al.. 2026. "Olympiad-level formal mathematical reasoning with reinforcement learning." *Nature*, 651, 8106, 607–613. ISBN: 1476-4687. https://doi.org/10.1038/s41586-025-09833-y.

G. Irving, P. Christiano, and D. Amodei. 2018. "AI safety via debate." *arXiv preprint arXiv:1805.00899*.

M. Jansen. 1981. "Maximal Nash subsets for bimatrix games." *Naval research logistics quarterly*, 28, 1, 147–152.

A. T. Kalai, E. Kalai, E. Lehrer, and D. Samet. 2010. "A commitment folk theorem." *Games and Economic Behavior*, 69, 1, 127–137.

V. Kovarík, C. Oesterheld, and V. Conitzer. 2023. "Game Theory with Simulation of Other Players." In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*. ijcai.org, 2800–2807. doi:10.24963/IJCAI.2023/312.

Y. Lin et al.. 2026. "Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction." In: *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. Ed. by C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust. proceedings.iclr.cc. https://proceedings.iclr.cc/paper%5C_files/paper/2026/hash/13e8be77982beb73d7ed0bbf122f9f3c-Abstract-Conference.html.

M. H. Löb. 1955. "Solution of a Problem of Leon Henkin." *The Journal of Symbolic Logic*, 20, 2, 115–118. doi:10.2307/2266895.

A. Mennen. 2013. *Prisoner's dilemma tournament results*. LessWrong. (2013). https://www.lesswrong.com/posts/QP7Ne4KXKytj4Krkx/prisoner-s-dilemma-tournament-results-0.

METR. Aug. 2026. *Brief Independent Investigation of Agents' Behavior, Reasoning and Collaboration in the OpenAI / Hugging Face Hacking Incident*. Tech. rep. METR, (Aug. 2026). https://metr.org/hugging-face-incident-report-aug-2026.pdf.

L. d. Moura and S. Ullrich. 2021. "The Lean 4 Theorem Prover and Programming Language." In: *Automated Deduction – CADE 28*. Ed. by A. Platzer and G. Sutcliffe. Springer International Publishing, Cham, 625–635. ISBN: 978-3-030-79876-5.

J. F. Nash. 1950. "Equilibrium points in n-person games." *Proceedings of the National Academy of Sciences*, 36, 1, 48–49.

J. F. Nash. 1951. "Non-Cooperative Games." *Annals of Mathematics*, 54, 2, 286–295.

C. Oesterheld. 2019. "Robust program equilibrium." *Theory and Decision*, 86, 1, 143–159.

C. Oesterheld, J. Treutlein, R. B. Grosse, V. Conitzer, and J. N. Foerster. 2023. "Similarity-based cooperative equilibrium." In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine.

OpenAI. Aug. 2026. *OpenAI–Hugging Face Incident Technical Report*. Tech. rep. OpenAI, (Aug. 2026). https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf.

P. Pudlák. 2017. "Incompleteness in the finite Domain." *Bull. Symb. Log.*, 23, 4, 405–441. doi:10.1017/BSL.2017.32.

P. Pudlák. 1986. "On the length of proofs of finitistic consistency statements in first order theories." In: *Logic Colloquium '84*. Studies in Logic and the Foundations of Mathematics. Vol. 120. Ed. by J. Paris, A. Wilkie, and G. Wilmers. Elsevier, 165–196. doi:https://doi.org/10.1016/S0049-237X(08)70462-2.

Z. Z. Ren et al.. 2025. *DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition*. (2025). https://arxiv.org/abs/2504.21801 arXiv: 2504.21801 [cs.CL].

S. Saito and M. Noguchi. 2026. *Formalized Formal Logic: A Mechanized Proof Library for Mathematical Logic*. Lean 4 library, Apache-2.0. Git commit a77e4e9, accessed September 2, 2026. (2026). https://github.com/FormalizedFormalLogic/Foundation.

M. Tennenholtz. 2004. "Program equilibrium." *Games and Economic Behavior*, 49, 2, 363–373.

The mathlib Community. 2020. "The Lean Mathematical Library." In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, 367–381. doi:10.1145/3372885.3373824.

H. Wang et al.. 2025. "Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning." *CoRR*, abs/2504.11354. arXiv: 2504.11354. doi:10.48550/ARXIV.2504.11354.

K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. 2023. "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models." In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine.

K. Zhang and D. Shasha. 1989. "Simple fast algorithms for the editing distance between trees and related problems." *SIAM journal on computing*, 18, 6, 1245–1262.

K. Zheng, J. M. Han, and S. Polu. 2022. "miniF2F: a cross-system benchmark for formal Olympiad-level mathematics." In: *International Conference on Learning Representations*. https://openreview.net/forum?id=9ZPegFuFTFv.

A Proofs

This Appendix collects all proofs from the main paper.

Proof of Theorem 2.4. Let $\boldsymbol{a}^* \in A \times A$ be a NE of G. Fix a player i and a deviation $\pi_i \in \Sigma$. Since the opponent's constant program always outputs a^*_{-i}, let $b_i := \mathrm{run}(\pi_i \parallel a^*_{-i}) \in A$. Then

$$U_i(\pi_i, a^*_{-i}) = u_i(b_i, a^*_{-i}) \leq u_i(a^*_i, a^*_{-i}) = U_i(a^*_i, a^*_{-i}),$$

where the inequality follows because $\boldsymbol{a}^*$ is a Nash equilibrium of G. Thus no unilateral program deviation is profitable, and $\boldsymbol{a}^*$ is a Program equilibrium. □

Proof of Theorem 2.9. Since constant programs ignore their input, $\mathrm{run}(a \| \mu) = a$ for all $a \in A$ and $\mu \in \Delta(\Sigma)$. In particular, $\mathrm{outcome}^\tau(\boldsymbol{a}^*) = \boldsymbol{a}^*$, and hence $U_i^\tau(\boldsymbol{a}^*) = u_i(\boldsymbol{a}^*)$ for each $i \in I$. Fix $i \in I$ and a deviation $\pi'_i \in \Sigma$, and set $b := \mathrm{run}\big(\pi'_i \parallel \tau_i(a^*_{-i})\big) \in A$. The opponent's constant program outputs a^*_{-i} whatever belief $\tau_{-i}(\pi'_i)$ it receives, so

$$U_i^\tau(\pi'_i, a^*_{-i}) = u_i(b, a^*_{-i}).$$

($\Leftarrow$) If $\boldsymbol{a}^*$ is a Nash equilibrium of G, then $u_i(b, a^*_{-i}) \leq u_i(\boldsymbol{a}^*) = U_i^\tau(\boldsymbol{a}^*)$, so π'_i is not a profitable deviation. Since i and π'_i were arbitrary, $\boldsymbol{a}^*$ is a τ-Program Equilibrium.

($\Rightarrow$) For every $b \in A$, the constant program b belongs to Σ and satisfies $U_i^\tau(b, a^*_{-i}) = u_i(b, a^*_{-i})$. If $\boldsymbol{a}^*$ is a τ-Program Equilibrium, then $u_i(b, a^*_{-i}) \leq U_i^\tau(\boldsymbol{a}^*) = u_i(\boldsymbol{a}^*)$ for all $i \in I$ and $b \in A$, that is, $\boldsymbol{a}^*$ is a Nash equilibrium of G. □

Theorem A.1. *Assume player $i \in I$ has undetectable deviations. If $\boldsymbol{\pi} \in \Sigma \times \Sigma$ is a pure τ-program equilibrium with outcome $\boldsymbol{a}$ then $a_i \in \mathrm{BR}_i(a_{-i})$, where BR_i is the best-response correspondence defined by*

$$\mathrm{BR}_i(a_{-i}) := \mathrm{argmax}_{a' \in A} u_i(a', a_{-i}).$$

Moreover, if both players have undetectable deviations, then the outcomes of pure τ-program equilibria are exactly the pure Nash equilibria of G.

Proof of Theorem A.1, hence of Theorem 2.11. Let $\boldsymbol{\pi}$ be a pure τ-PE with outcome $\boldsymbol{a}$. Define $\mu := \tau_i(\pi_{-i})$ and let $x \in \mathrm{BR}_i(a_{-i})$. Let ρ satisfy Definition 2.10 for (π_i, μ, x). Then by 2.

$$\mathrm{run}(\rho \| \mu) = x,$$

and by 1.

$$\mathrm{run}(\pi_{-i} \| \tau_{-i}(\rho)) = \mathrm{run}(\pi_{-i} \| \tau_{-i}(\pi_i)) = a_{-i}.$$

The equilibrium condition then implies that

$$u_i(a_i, a_{-i}) = U_i^\tau(\boldsymbol{\pi}) \geq U_i^\tau(\rho, \pi_{-i}) = u_i(x, a_{-i}) = \max_{a' \in A} u_i(a', a_{-i}).$$

Finally, if both players can mimic, applying this argument to $i = 1$ and $i = 2$ shows that $\boldsymbol{a}$ is a pure Nash equilibrium of G. Conversely, by Theorem 2.9, every pure Nash equilibrium of G is the outcome of a pure τ-PE, namely the profile of the corresponding constant programs. □

B Formal Tools and Theoretical Results in Open-Source Game Theory

This appendix states in detail the definitions, outcomes and proofs sketched in Section 3.2 (Appendix B.1), then presents our corrections and progress on Critch et al. (2022) (Appendix B.3).

B.1 Mathematical Tools and Outcomes

We take our running example `DupocBot` and illustrate each tool using concrete matchups of this bot.

B.1.1 Soundness. The first important property of our system S is that it is sound, i.e. that proved statements are true.

Proposition B.1 (Soundness). *Every sentence provable in S is true in $\mathbb{N}$ under the standard reading. In particular, $\vdash_k \varphi$ implies $\models \varphi$, for every budget k.*

Proof. $\mathbb{N}$ satisfies the axioms of Peano Arithmetic, and under the standard reading, where c_C is 0 and c_D is 1, it satisfies the action axiom. So $\mathbb{N}$ is a model of T_{Act}, and every sentence a theory proves holds in each of its models (Soundness of first-order theories; Enderton, 2001, Section 2.5). □

Soundness allows us to prove that certain searches must fail, through its contrapositive:

Proposition B.2. *Let* `DefectBot` *be the program that always defects. Then for every k,*

$$\mathrm{outcome}(\texttt{DupocBot}(k), \texttt{DefectBot}) = (D, D)$$

Proof. `DefectBot` plays D, so the hypothesis "`DefectBot` cooperates with me" is false. By the contrapositive of Proposition B.1 it has no proof, of any length. The search of `DupocBot`(k) therefore fails at every budget, and `DupocBot`(k) plays its default, D. □

Soundness also ensures robustness of certain programs, which we call *unexploitable*: a program π such that no opponent π' can get $\mathrm{outcome}(\pi, \pi') = (C, D)$. Programs that cooperate *only after proving that their opponent cooperates with it* have this property, because the proof they found guarantees (thanks to soundness) the opponent's cooperation.

Proposition B.3. `DefectBot`, `DupocBot` *and* `CIMCIC`[23] *are unexploitable.*

Proof. `DefectBot` never cooperates, so there is nothing to show.

`DupocBot(k)` only cooperates if `proof_search(k, opp, φ)` succeeded for φ = "`opp` cooperates against `DupocBot(k)`". So if `DupocBot` cooperates, that means φ is provable, which means its opponent playing C is true, by soundness.

Similarly, `CIMCIC(k)` only cooperates if `proof_search(k, opp, φ)` succeeded for φ = "if `CIMCIC(k)` cooperates, then `opp` cooperates against it". If `CIMCIC` cooperates, that means φ is provable, thus true, by soundness. Since the premise of φ is also true, then the conclusion must be true, i.e its opponent plays C. □

B.1.2 Completeness. Naturally, besides soundness, S is complete: if a statement is true for any model of S, then S should be able to prove it:

Theorem B.4 (Completeness, Gödel (1930)). *Let T be a first-order theory and φ a sentence of its language. If φ holds in every model of T, then T proves φ. In other words, if $M \models \varphi$ for every model M of T, then $T \vdash \varphi$, with no bound on the length of the proof.*

For S, checking every model reduces to checking the two readings of the constants:

Corollary B.5 (Completeness over the two readings). *A sentence that holds in every model of Peano Arithmetic, under both the standard and the swapped reading of c_C and c_D, is provable in S.*

Proof. By the action axiom, the models of T_{Act} are exactly the models of Peano Arithmetic equipped with one of the two readings. The claim is therefore Theorem B.4 applied to T_{Act}. □

This second tool allows us to show that a search succeeds. Against `CooperateBot`, the program that always plays C, the hypothesis of `DupocBot`(k) is true, and `DupocBot`(k) cooperates exactly if it has a proof of length at most k. Instead of producing such a proof, we show that one exists, using Gödel's completeness theorem:

Proposition B.6. *For all sufficiently large k,*

$$\text{outcome}(\texttt{DupocBot}(k), \texttt{CooperateBot}) = (C, C)$$

Proof. Let $G[k]$ be the hypothesis "`CooperateBot` cooperates with `DupocBot`(k)". The sentence $\forall k\, G[k]$ holds in every model of Peano Arithmetic under both readings: under the standard one it says that a constant program plays its constant, and under the swapped one it says the same of the transposed programs. By Corollary B.5, S proves $\forall k\, G[k]$, with a proof of some fixed length N. Instantiating it at a given k adds the length of the numeral of k, which we can write in logarithmic terms, so $\vdash_k G[k]$ as soon as k exceeds N plus this overhead. The search of `DupocBot`(k) then succeeds and it plays C, as does `CooperateBot`. □

Completeness gives a proof without giving its length. The outcome therefore holds for all k above a threshold that is shown to exist but is not computed, whereas soundness involves no estimate on lengths and gives outcomes at *every* budget. Every outcome in this paper is of one of these two kinds. Neither, however, handles self-reference.

B.1.3 Löb's Theorem. When `DupocBot`(k) faces itself, its hypothesis is true only if its own search succeeds (Example 2.6). Neither of the tools above help, because we cannot verify the truthfulness of the hypothesis before knowing the result of the search. This self-fulfilling fact is directly resolved by Löb's theorem (Löb 1955), which we state in its bounded form:

[23] See Appendix G for programs' definitions.

Theorem B.7 (Bounded Löb, after Critch (2019); see also Critch et al. (2022), Lemma 3.6). *Let $\varphi[k]$ be a formula of S with a single free variable k. Suppose that, for some $k_1 \in \mathbb{N}$, S can verify that $\varphi[k]$ is "potentially self-fulfilling for large k":*

$$\vdash \left(\forall\, k > k_1\right)\left(\Box_k(\varphi[k]) \to \varphi[k]\right).$$

Then there is a threshold $k_2 \in \mathbb{N}$ such that $\vdash_k \varphi[k]$ for every $k > k_2$.

Critch (2019) derive this theorem from four efficiency assumptions on the proof system, recalled in Critch et al. (2022, Appendix B). For S none of them is assumed. Property (a), that S can express every computable function, holds because S extends Peano Arithmetic. Property (b), that S writes the number k in $O(\lg k)$ symbols, holds by the choice of binary numerals (appendix C). The abbreviation mechanism (c) is not used. The assumption (d) that a proof of length ℓ can be checked inside the system by a proof of length linear in ℓ is replaced by a theorem with a polynomial bound: if $\vdash_\ell \varphi$, then S proves $\Box_\ell\varphi$ with a proof of length polynomial in ℓ[24]. This weaker form suffices for Theorem B.7.

Theorem B.8 (Critch et al. (2022), Thm. 3.7). *For all sufficiently large k*

$$\text{outcome}(\texttt{DupocBot}(k), \texttt{DupocBot}(k)) = (C, C)$$

Proof. Apply Theorem B.7 with $\varphi[k]$ = "`DupocBot(k)(DupocBot(k)) == C`" and $k_1 = 0$. If $\varphi[k]$ has a proof of length at most k, then $\texttt{DupocBot}(k)$ finds it and plays C, so $\Box_k(\varphi[k]) \to \varphi[k]$ holds. This reasoning is valid in every model of Peano Arithmetic and under both readings, so the implication is provable in S by corollary B.5. Theorem B.7 then gives $\vdash_k \varphi[k]$ for every $k > k_2$: the search of each copy of $\texttt{DupocBot}(k)$ succeeds, and both play C. □

B.2 Proofs of Section 3.3

This Appendix contains proofs and additional details on the material covered in Section 3.3 and the proof of `DupocBot` vs. `CupodBot`.

Proof of Theorem 3.3. Let d be a proof of φ of length at most k, and apply $\mathcal{T}$ to every formula of d. We check that the result $\mathcal{T}d$ is a proof of $\mathcal{T}\varphi$. A logical step remains a logical step, because $\mathcal{T}$ commutes with the logical symbols and logical rules do not depend on which constant occurs. An axiom of Peano Arithmetic does not mention the two constants and is unchanged. The action axiom is sent to itself with its two disjuncts exchanged, which is again an axiom by (S2). Hence $\mathcal{T}d$ is a proof of $\mathcal{T}\varphi$. It has the same length as d, since the length of a proof counts its symbols without regard to which symbol they are. This gives $\vdash_k \varphi \Rightarrow \vdash_k \mathcal{T}\varphi$, and the converse follows by applying the same argument to $\mathcal{T}\varphi$, as $\mathcal{T}$ is an involution. □

We move on to proving theorem 3.5, which demands careful attention as to how programs are written inside a sentence. Consider the sentence of section 3.3

$$\varphi \;=\; \text{Plays}(\pi, \rho, a),$$

"π plays a against ρ". Programs are encoded by natural numbers by an injective coding, and we write $\text{code}(\pi)$ for the code of π (it's value in $\mathbb{N}$). Since the language of S is $\{0, 1, +, \cdot, <, c_C, c_D\}$, the sentence φ can mention π only through a closed term that evaluates to $\text{code}(\pi)$, which we call a name of π and write $\overline{\pi}$.

The question of which term to use for the name arises because of what $\mathcal{T}$ must do to φ. Theorem 3.5 asks that $\mathcal{T}\varphi$ be the sentence "$\mathcal{T}\pi$ plays $\mathcal{T}a$ against $\mathcal{T}\pi'$". Now suppose we take the obvious name, the numeral of $\text{code}(\pi)$. It is a string of 0, 1 and +, which $\mathcal{T}$ leaves unchanged, so $\mathcal{T}\varphi$ flips only the constant c_a and reads "π

[24] This is further explained in appendix C, `boundedInnerNec_sixteen`.

plays $\mathcal{T}a$ against π'": the same two programs, the other action. For the theorem to hold, the names themselves must contain the two constants, placed so that exchanging them turns the name of π into the name of $\mathcal{T}\pi$,

$$\mathcal{T}(\overline{\pi}) \;=\; \overline{\mathcal{T}\pi}.$$

We now construct names with this property.

First, define $\mathrm{reval}(u, w, x)$, a function that, for a program code x and $u, w \in \{0, 1\}$, replaces every action value 0 of the program by u and every action value 1 by w, including the action constants inside the hypotheses the program stores, leaving the rest of its syntax unchanged, and returns a fixed default on other inputs. Thus, for every program π,

$$\mathrm{reval}(0, 1, \mathrm{code}(\pi)) = \mathrm{code}(\pi),$$
$$\mathrm{reval}(1, 0, \mathrm{code}(\pi)) = \mathrm{code}(\mathcal{T}\pi).$$

To construct the names, we choose a common representative code for each pair $\{\pi, \mathcal{T}\pi\}$,

$$\mathrm{rep}(\pi) := \min\{\mathrm{code}(\pi), \mathrm{code}(\mathcal{T}\pi)\},$$

Since $\mathcal{T}$ is an involution, $\mathrm{rep}(\mathcal{T}\pi) = \mathrm{rep}(\pi)$. We then choose the arguments that reconstruct π from this representative:

$$(u_\pi, w_\pi) := \begin{cases} (0, 1), & \text{if } \pi = \mathcal{T}\pi, \\ (c_C, c_D), & \text{if } \mathrm{code}(\pi) < \mathrm{code}(\mathcal{T}\pi), \\ (c_D, c_C), & \text{if } \mathrm{code}(\mathcal{T}\pi) < \mathrm{code}(\pi). \end{cases}$$

Here 0 and 1 are numerals. Injectivity of the coding makes the three conditions exclusive and exhaustive.

Definition B.9 (Names of programs). The name of a program π is the closed term

$$\overline{\pi} := \mathrm{reval}\big(u_\pi, w_\pi, \dot{\mathrm{rep}}(\pi)\big).$$

Under the standard reading, $\overline{\pi}$ evaluates to $\mathrm{reval}(0, 1, \mathrm{rep}(\pi))$ or $\mathrm{reval}(1, 0, \mathrm{rep}(\pi))$ according to the case, which is $\mathrm{code}(\pi)$ in each; under the swapped reading it evaluates to $\mathrm{code}(\mathcal{T}\pi)$.

We use these names to express statements about program execution. Let $\mathrm{Run}(x, y, z)$ be a fixed arithmetical formula asserting that the program with code x, run on input y, halts with output z. For programs π, ρ and an action a, the sentence of section 3.3 is

$$\mathrm{Plays}(\pi, \rho, a) := \mathrm{Run}(\overline{\pi}, \overline{\rho}, c_a).$$

The formula Run contains no action constant; the constants enter the sentence only through the three substituted terms.

Proof of theorem 3.5. We first show what was promised: $\mathcal{T}(\overline{\pi}) = \overline{\mathcal{T}\pi}$. We have $\mathrm{rep}(\mathcal{T}\pi) = \mathrm{rep}(\pi)$, and

$$(\mathcal{T}u_\pi, \mathcal{T}w_\pi) = (u_{\mathcal{T}\pi}, w_{\mathcal{T}\pi}),$$

because $\mathcal{T}$ exchanges the two orientations when $\pi \neq \mathcal{T}\pi$ and fixes the pair of numerals $(0, 1)$ when $\pi = \mathcal{T}\pi$. Since $\mathcal{T}$ also fixes the defining formula of reval and the term $\dot{\mathrm{rep}}(\pi)$, the identity follows, and likewise for ρ. Since $\mathcal{T}$ fixes Run and $\mathcal{T}c_a = c_{\mathcal{T}a}$,

$$\mathcal{T}\,\mathrm{Plays}(\pi, \rho, a) = \mathrm{Run}(\mathcal{T}(\overline{\pi}), \mathcal{T}(\overline{\rho}), \mathcal{T}(c_a)) = \mathrm{Run}(\overline{\mathcal{T}\pi}, \overline{\mathcal{T}\rho}, c_{\mathcal{T}a}) = \mathrm{Plays}(\mathcal{T}\pi, \mathcal{T}\rho, \mathcal{T}a)$$

as an identity of syntactic expressions. □

Proof of theorem 3.6. Fix k, and write $\pi = \texttt{DupocBot}(k)$ and $\rho = \texttt{CupodBot}(k) = \mathcal{T}\pi$. Their searches have the respective hypotheses

$$\varphi := \text{Plays}(\rho, \pi, C), \qquad \psi := \text{Plays}(\pi, \rho, D).$$

By theorem 3.5 and $\mathcal{T}^2\pi = \pi$, $\psi = \mathcal{T}\varphi$, so theorem 3.3 gives $\vdash_k \varphi \Leftrightarrow \vdash_k \psi$: the two searches either both succeed or both fail. If both succeed, π plays C and ρ plays D; but a found φ is true by soundness, so ρ plays C, a contradiction. Hence both fail, and each program plays its default: π plays D and ρ plays C. □

B.3 Corrections and Progress on Critch et al. (2022)

With a concrete formal system and the three tools above, we can revisit the open problems of Critch et al. (2022). Löb's theorem alone turns out to be enough to refute one of their stated examples, and the tools together resolve or reduce several of their open problems. We begin with the refutation, then present the open problems and what the library proves about each.

B.3.1 `DIMCID` *Always Defects: A Refutation of Example 5.3(a).* Consider the bot `DIMCID`: "defect if my cooperation implies your defection" (Critch et al. 2022, §5.2). Below, we prove, using Löb's theorem, that `DIMCID` defects against every closed program, so that it is behaviorally equivalent to `DefectBot`. This directly contradicts Example 5.3(a) of Critch et al. (2022), where they stipulate that `DIMCID` cooperates with `CooperateBot`

Theorem B.10 (`DIMCID` always defects). *For every program π there is a k_0 such that for all $k > k_0$,*

$$\text{outcome}\big(\texttt{DIMCID}(k), \pi\big) = (D, \cdot),$$

and the guard of $\texttt{DIMCID}(k)$ *is provable in S within length k.*

Proof. Write p for "$\texttt{DIMCID}(k)$ defects against π" and q for "π defects against $\texttt{DIMCID}(k)$". The bot searches for a proof of $\neg p \to q$, which is $p \vee q$, and defects exactly when it finds one, so its action is the fixed point of

$$p \;\leftrightarrow\; \Box_k(p \vee q).$$

Since $p \to p \vee q$, we have $\Box_k p \to \Box_k(p \vee q)$, and the fixed point turns the right-hand side into p; hence $\Box_k p \to p$. Löb's Theorem B.7 then gives p, for large k, whatever q is. In words, a proof of the guard makes the bot defect, which falsifies the antecedent $\neg p$, which makes the guard true, and the opponent never enters. □

B.3.2 Progress on Some Open Problems. Apart from our novel proof on `DupocBot` vs. `CupodBot`, we present three other open problems of Critch et al. (2022) are addressed by the library. One is settled, one reduced to bounded-consistency questions, and one answered by construction

Open Problem 5: $\text{outcome}(\texttt{CUPOD}(k), \texttt{CIMCIC}(k))$. Critch et al. (2022) rules out (D, C), since `CupodBot` never exploits, and leaves (C, C), (C, D) and (D, D) open. In S the cell is a genuine floor with two nested hinges (definition 4.1): G_k, "$\texttt{CIMCIC}(k)$ proves within length k that its cooperating implies $\texttt{CupodBot}(k)$'s", and U_k, "$\texttt{CupodBot}(k)$ proves within length k that $\texttt{CIMCIC}(k)$ defects". We prove, for all large k, that G_k gives (C, C), that $\neg G_k \wedge U_k$ gives (D, D), and that $\neg G_k \wedge \neg U_k$ gives (C, D). These are exactly Critch's three cases, so the open problem is not closed but reduced: each hinge is a bounded-consistency question, and the floor side of our matrix, which takes both hinges negative, reports (C, D), both searches failing.

Open Problem 6: $\text{outcome}(\texttt{DUPOC}(k), \texttt{DIMCID}(k))$. Settled, with no hinge: (D, D) for all large k. By theorem B.10, $\texttt{DIMCID}(k)$ always defects; $\texttt{DupocBot}(k)$'s guard, "$\texttt{DIMCID}(k)$ cooperates with me", is then false in $\mathbb{N}$, so by soundness no proof of it exists at any length and $\texttt{DupocBot}(k)$ defects.

Open Problem 9. A computationally bounded `PrudentBot`: "Cooperates iff I prove within length k both that my opponent cooperates with me and that my opponent defects against `DefectBot`." It exists, and its row is in the library (table 3).

Table 3. `PrudentBot`(k)'s row against every program of the paper in S, for all large k. ‡ is the hinge dichotomy: the first row gives the negative side ($\neg H_k$, the one the matrices of section 5 use), the second the positive side (H_k). Green: mutual cooperation; gray: mutual defection.

	Coop	Cup	DB	Def	Dup	EB	OB	TFT	CupT	DupT	CIM	DID	DIMCID	Prud
negative side	DC	DC‡	DC	DD	DD‡	DC‡	DD	DD	DC	DC	DD‡	DC‡	DD	DD‡
positive side		DD			CC	CC					CC	DD		CC

C The Lean Formalization of S

In this appendix, we detail the Lean formalization of S summarized in section 4.1.3 (section C.1) and record how long each of its parts took (section C.2).

Table 4. The trusted base: the nine definitions on which every result rests. The file `Tour.lean` of the repository restates all nine at $\mathbb{N}$, each kept in sync with its definition by the compiler, and walks through the headline theorems; it is the recommended entry point into the code.

Definition	What it fixes	Lean
Language and theory	arithmetic with c_C, c_D and the action axiom	`LAct`, `TAct`
Proof length	the symbol count of a derivation	`proofLen`
The box	$\Box_k$, bounded provability as a formula	`LenBox`
Program codes	programs as numbers, by a polynomial pairing	`progCode`
Names	a program named by a short term	`codeTerm`, `descriptions`
Templates	a hypothesis as a formula with open slots	`template`
The guard	the sentence a search consults	`guardCode`
Substitution	simulated programs read the outer players	`substCode`
The evaluator	what a program plays, as a relation in arithmetic	`EvalGraph`

C.1 Proof Length, Encodings and Bounded Provability

Proof length. A proof of S is a derivation in the sequent calculus of the Foundation library (Saito and Noguchi 2026), a one-sided calculus with cut: a tree in which every node carries a sequent, that is, a finite set of formulas. For a derivation d, we define its length $|d|$ as

$$|d| \;=\; \sum_{\text{nodes } n \text{ of } d} \big(1 + |\Gamma_n|\big) \;+\; \sum_{\exists\text{-introductions}} |t|,$$

where $|\Gamma_n|$ is the number of symbols in the sequent at node n and $|t|$ the number of symbols in the witness term of an $\exists$-introduction. Two properties are at the core of this design. First, it is symbol-blind: every symbol counts for one, whichever symbol it is. The length of a proof is therefore unchanged by any renaming of symbols, in particular by τ; together with the action axiom being stated in both orders, this is what makes theorem 3.3 exact, with no overhead in the budget. Second, it is proper: every node pays for its whole sequent and every witness is paid for, so only finitely many proofs have length at most k, with an explicit bound on their codes. Since a node pays for its sequent, a proof is at least as long as the sentence it proves:

$$\vdash_k \varphi \quad\Longrightarrow\quad |\varphi| \le k.$$

Encodings. S is a theory of arithmetic: its sentences speak about natural numbers and nothing else, since its signature contains only 0, 1, +, ·, < and the two action constants. For S to reason about sentences, proofs and programs, each of these objects is assigned a natural number, its Gödel code, written $\ulcorner\cdot\urcorner$. For example, programs are encoded using the following pairing:

$$\ulcorner\texttt{.search}\; k\; \varphi\; p\; q\urcorner \;=\; \langle 6, \langle k, \langle \ulcorner\varphi\urcorner, \langle \ulcorner p\urcorner, \ulcorner q\urcorner\rangle\rangle\rangle\rangle + 1, \qquad \text{where} \quad \langle x, y\rangle := (x+y)^2 + y,$$

which is injective. The code of a program is then the value of a term built from the numerals of its leaves, and a program of budget k can be named inside a sentence by a short expression in which k appears as a binary numeral, of length $O(\log k)$. This is assumption (b) of Critch et al. (2022).

Bounded provability. With proofs coded as numbers, "d is a proof of the sentence coded by x" is an arithmetical relation $\mathrm{Proof}_{T_{\mathrm{Act}}}(d, x)$, provided by the Foundation library, and bounded provability is the formula

$$\Box_k(x) \;:=\; \exists\, d < B(k).\;\; \mathrm{Proof}_{T_{\mathrm{Act}}}(d, x) \;\wedge\; |d| \le k,$$

read "some proof of the sentence coded by x has length at most k"; we write $\Box_k\varphi$ for $\Box_k(\ulcorner\varphi\urcorner)$. The bound $B(k)$ is the bound on the codes of proofs of length at most k that properness provides, so that the quantifier ranges over a finite set and $\Box_k$ is a bounded search. In Lean:

```
def LenBox (k φ : V) : Prop := ∃ d < codeBound k, Proof T d φ ∧ proofLen T d ≤ k
```

From the proofs to Lean. Every result of section 3 is mechanized; table 5 gives the name of the most impactful ones.

Table 5. The results of Appendix B and their Lean counterparts.

Result	Lean
Soundness (proposition B.1)	`TAct_sound_nat`
Completeness over the two readings (corollary B.5)	`TAct_complete_of_readings`
Bounded Löb (theorem B.7)	`family_found`
Transposition of provability (theorem 3.3)	`lenBoxNat_swap_iff`
Transposition of statements about programs (theorem 3.5)	`lMap_swap_sentenceAt_plays`
Transposition of outcomes (theorem 3.5)	`outcomeSpecA_transpose`

One important fact about Theorem B.7 is that it rests on S being able to verify its own proofs at bounded cost, and to know that it can: if $\varphi[k]$ has a proof of length at most ℓ, then $\Box_\ell\varphi[k]$ has a proof of length at most $P(\ell)$, and this implication holds in every model of S, uniformly in k, so that S proves it. This is the inner necessitation of provability logic, with lengths attached. Critch et al. (2022) assumes P to be linear (Appendix B, assumption (d)); we prove it for P polynomial, of degree 16. The weaker bound suffices because the Löb argument applies it only to proofs of length polylogarithmic in the budget k, which a polynomial keeps far below k. The proof is `boundedInnerNec_sixteen`; section C.2 gives its share of the development.

C.2 Development Effort

This subsection records how long each part of the formalization of section 4 took. As stated in section 4.1, the authors designed the definitions, statements and architecture, and LLM agents wrote most of the Lean source. Table 6 lists the steps in order. Hours are estimated from the commit timestamps of the repository: a run of commits less than two hours apart counts as continuous work. They measure elapsed time during which agents were producing commits, not agent-hours, since sessions may have run in parallel and we did not record how

many. The formalization comprises about 160,000 lines of Lean, of which the proof of bounded inner necessitation

Table 6. Estimated development effort of the formalization of S, in hours of agent activity.

Step	Hours	Result
S and conditional Löb	≈ 25	Language, theory and proof length; program codes; the diagonal lemma; the bounded Löb theorem and `DupocBot`'s self-cooperation, conditional on bounded inner necessitation
Bounded inner necessitation	≈ 40	The hypothesis is proved, so both results become unconditional
Outcome matrix	≥ 10	107 outcome cells, and the 26 open cases stated as dichotomies
Partial transparency	≈ 10	The three τ constructors in programs and evaluator; the τ-rows of 14 templates

accounts for more than half. In the outcome matrix, cells proved by a uniform method (evaluation, soundness, completeness) came in batches of ten to thirty, and Löb cells one or a few at a time; because they were committed in batches, the figure for that step is a lower bound.

D Equilibrium Computation and Selection Dynamics

In this Appendix, we describe our procedures for enumerating and classifying mixed program equilibria and for analyzing the long run dynamics of the program meta-game. The same methods apply to the full-transparency program meta-game G^{prog} of Section 2.1 and the partial-transparency meta-game G^{τ} of Section 2.2, with the program library and transparency maps fixed before each computation. All equilibrium claims are relative to the chosen program library.

D.1 Enumerating Program Equilibria

Let $G = (U, U^{\top})$ be a finite symmetric two players game, where U denotes the payoff matrix of the row player against the column player; as usual, denote by A the (common) action set. We enumerate its extreme Nash equilibria using `lrsNash` in exact rational arithmetic (Avis et al. 2010). If the payoff matrix U had negative entries, a common additive shift (the minimum value) makes all payoffs positive without changing best responses. The algorithm enumerates completely labeled vertex pairs of the associated best-response polytopes and normalizes them to mixed-strategy profiles. Complete labeling expresses the equilibrium condition that each pure strategy is either assigned zero probability or is a best response to the opponent's mixture.

The finite list of extreme equilibria describes the entire equilibrium set, including any continua. Specifically, the Nash equilibrium set of a finite two-players game is a finite union of maximal Nash subsets of product form (Jansen 1981):

$$\mathrm{NE}(G) = \bigcup_{k=1}^{K} C_k, \qquad C_k = X_k \times Y_k \subseteq \Delta(A) \times \Delta(A),$$

where X_k and Y_k are convex polytopes. Writing

$$X_k = \mathrm{conv}\{\xi_{k,1}, \ldots, \xi_{k,r_k}\}, \qquad Y_k = \mathrm{conv}\{\eta_{k,1}, \ldots, \eta_{k,s_k}\},$$

the vertices of C_k are precisely the extreme equilibrium profiles $(\xi_{k,i}, \eta_{k,j})$. Consequently, every profile

$$\left(\sum_{i=1}^{r_k} \lambda_i \xi_{k,i}, \sum_{j=1}^{s_k} \mu_j \eta_{k,j} \right), \qquad \text{for } \lambda_i, \mu_j \geq 0, \text{ and } \sum_i \lambda_i = \sum_j \mu_j = 1,$$

is a Nash equilibrium. The convex weights for the two players may be chosen independently.

To assign connected-component labels, we construct a bipartite graph whose two vertex classes are the distinct row and column strategies appearing in the enumeration, with an edge for each extreme equilibrium. Its maximal complete bipartite subgraphs determine the maximal Nash subsets, and its connected components determine the connected components of $\mathrm{NE}(G)$.

We apply this procedure in turn to the full transparency special case (Section 2.1), corresponding to G^{prog}, and the partial transparency meta-game (Section 2.2), corresponding to G^{τ}. The action sets correspond to the program libraries Σ.

D.2 Replicator Dynamics

For each connected component of $\mathrm{NE}(G)$ identified in Appendix D.1, we estimate the probability that a population with uniformly drawn initial composition converges to that component. Let $\boldsymbol{\alpha} \in \Delta(A)$ be a mixed action, under the mass action interpretation, and let α_i denote the share of the population playing action $i \in A$. Under random matching, $(U\boldsymbol{\alpha})_i$ is the expected payoff to i and $\boldsymbol{\alpha}^\top U \boldsymbol{\alpha}$ is the population mean. The replicator dynamics are

$$\dot{\alpha}_i = \alpha_i \cdot g_i(\boldsymbol{\alpha}), \qquad g_i(\boldsymbol{\alpha}) = (U\boldsymbol{\alpha})_i - \boldsymbol{\alpha}^\top U \boldsymbol{\alpha}.$$

Thus a program present in the population gains share whenever its payoff exceeds the mean. If an interior trajectory converges to $\boldsymbol{\alpha}^*$, then $(\boldsymbol{\alpha}^*, \boldsymbol{\alpha}^*)$ is a program equilibrium, see Hofbauer and Sigmund (2003).

We draw $1,000$ initial states independently and uniformly from $\Delta(A)$ using a Dirichlet$(1, \ldots, 1)$ distribution. At and $T = 10^5$ and $T/2$, we label action i as absent if $\alpha_i < \varepsilon$ and tied with the mean if $|g_i(\boldsymbol{\alpha})| \le \varepsilon$, where $\varepsilon = 10^{-6}$. We classify a trajectory as converged if, at both times, no action exceeds the mean by more than ε, every action is absent or tied, both label sets agree across times, and $\|\boldsymbol{\alpha}(T) - \boldsymbol{\alpha}(T/2)\|_\infty < \varepsilon$. Using the exact enumeration from Appendix D.1, we retain the extreme strategies that assign zero share to every action labeled absent and against which every action labeled tied is a best response. Their convex hull consists of mixtures inducing symmetric equilibria in a single component. We assign the trajectory to that component after verifying that $\boldsymbol{\alpha}(T)$ is within 10^{-4} in supremum norm of a point in the hull.

D.3 Outcome Matrix under Full Transparency

Table 7 reports the pairwise outcomes of the twelve programs defined in Appendix G, with the row program's action listed first. Outcomes hold for sufficiently large shared proof budgets and sufficient evaluation fuel, subject to the indicated hinge assumptions. Applying the base-game payoffs to these outcomes yields the payoff matrices used for equilibrium enumeration and replicator dynamics.

E Results on Full Transparency

E.1 An Enumeration of Program Equilibria

Each row of Table 8 gives a strategy for either player, with probabilities listed in the same order as the bots. Any two rows in the same group, one for each player, give an extreme equilibrium with the probability of mutual cooperation shown in the group heading. For example, the four rows in the full-cooperation group give $4 \times 4 = 16$ extreme equilibria, including the four in which both players choose the same row.

In the underlying Prisoner's Dilemma, the unique Nash equilibrium is (D, D). In G^{prog}, the three components of $\mathrm{NE}(G^{\mathrm{prog}})$ (the set of program equilibria) yield mutual cooperation with probabilities 1, 11/18, and 0, respectively, in the order shown in Table 8. Thus full cooperation, with payoff $R = 2$ for each player, holds throughout an entire component, while the intermediate rate occurs at a single symmetric equilibrium.

For the chosen library Σ, `DupocBot` is the only program that both players use in every fully cooperative equilibrium. To see this, write the fully cooperative component as $C_1 = X_1 \times Y_1$. Each vertex of X_1 or Y_1 assigns

Table 7. The full outcome matrix of all the bots studied in this paper. Proved in S, at a shared proof budget k; a cell gives the row bot's action first. Cells hold for every sufficiently large k and every sufficient fuel. Shaded cells: mutual cooperation (green) and mutual defection (grey). Orange cells‡ rest on the hinge of definition 4.1, where we show its negative side. Highlighted names (yellow) are empirically unexploitable bots (proposition B.3).

	CooperateBot	DefectBot	TitForTatBot	DBot	OBot	EBot	DupocBot	CupodBot	CupodTrollBot	DupocTrollBot	CIMCIC	DIMDID
CooperateBot	CC	CD	CC	CD	CC	CD	CC	CC	CC	CC	CC	CC
DefectBot	DC	DD	DD	DC	DD	DD	DD	DD	DC	DC	DD	DD
TitForTatBot	CC	DD	CC	DC	CD	DC	CC	CC	CC	CC	CC	CC
DBot	DC	CD	CD	DD	CD	CD	CD‡	CC	DC	DC	CD‡	CC
OBot	CC	DD	DC	DC	DD	DC	DD	DD	CC	CC	DD	DD
EBot	DC	DD	CD	DC	CD	CC	CD‡	CC	DC	DC	CD‡	CC
DupocBot	CC	DD	CC	DC‡	DD	DC‡	CC	DC	DC‡	DD	CC	DC‡
CupodBot	CC	DD	CC	CC	DD	CC	CD	DD	DD	CC	CD‡	DD
CupodTrollBot	CC	CD	CC	CD	CC	CD	CD‡	DD	CC	CC	CD‡	CC
DupocTrollBot	CC	CD	CC	CD	CC	CD	DD	CC	CC	CC	CD‡	CC
CIMCIC	CC	DD	CC	DC‡	DD	DC‡	CC	DC‡	DC‡	DC‡	CC	DC
DIMDID	CC	DD	CC	CC	DD	CC	CD‡	DD	CC	CC	CD	DD

Table 8. All 21 extreme Nash equilibria of the restricted 8-bot program meta-game at full transparency (floor case).

Strategy support (either player)	Probabilities in support order
Full cooperation (16 extreme equilibria)	*Pr[(C, C)] = 1*
DupocBot	1
CooperateBot, DupocBot	2/3, 1/3
DupocBot, TitForTatBot	1/3, 2/3
CooperateBot, DupocBot, TitForTatBot	2/3, 1/9, 2/9
Partial cooperation (1 equilibrium)	*Pr[(C, C)] = 11/18*
CooperateBot, OBot, EBot	1/2, 1/3, 1/6
Full defection (4 extreme equilibria)	*Pr[(C, C)] = 0*
DefectBot	1
DefectBot, OBot	2/3, 1/3

probability at least 1/9 to DupocBot, so the same bound holds for both players at every $(\xi, \eta) \in C_1$ by convexity. No other program is required, since both players may use DupocBot alone.

E.2 Replicator Dynamics at Full Transparency

For the eight-program library Σ at full transparency (floor case), we drew 1,000 initial population states uniformly from $\Delta(\Sigma)$. All 1,000 trajectories met the convergence criterion of Appendix D.2, and all were assigned to the fully cooperative component C_1 of Table 8. Its estimated basin share is therefore 1, with a 95% Wilson interval of $[0.9962, 1]$. No trajectory was assigned to either the isolated equilibrium or the full-defection component. For each of these components, the estimated basin share is 0, with a 95% Wilson interval of $[0, 0.0038]$. Table 9 shows how the final population composition varied within C_1. DupocBot was present in every run. Its final share ranged

Table 9. Final population composition in the eight-program full-transparency game (floor case). A program is counted as present if its share at time T is at least $\varepsilon = 10^{-6}$.

Programs present at time T	Number of runs
CooperateBot, DupocBot, TitForTatBot	904
DupocBot, TitForTatBot	46
DupocBot	40
CooperateBot, DupocBot	10

Table 10. Equilibrium outcomes across program libraries at full transparency. The cooperative support lists the programs used in at least one fully cooperative equilibrium, with $S = \{\texttt{CooperateBot}, \texttt{DupocBot}, \texttt{TitForTatBot}\}$. The last column gives the fraction of $1,000$ uniformly drawn initial populations assigned to the fully cooperative component.

	Nash equilibria			Replicator dynamics
Library	Components	Extreme equilibria	Cooperative support	Cooperative share
$\Sigma \setminus \{\texttt{CupodBot}\}$	3	21	S	100%
Σ	3	21	S	100%
Σ + trolls	3	35	S	94.8%
Σ + trolls + provers	3	69	$S \cup \{\texttt{CIMCIC}\}$	100%

from 0.1697 to 1, compared with the lower bound of $1/9$ over the entire component derived in Appendix E.1. In 904 runs, both `CooperateBot` and `TitForTatBot` were also present. Despite this variation, every final population yielded mutual cooperation with probability 1 and mean payoff $R = 2$. Thus, for this library at full transparency, mutual cooperation is the predominant outcome of rational play when equilibrium selection follows the replicator dynamics from a uniformly drawn initial population.

E.3 Sensitivity to the Program Library

We repeat the full-transparency analysis of Appendix D for four nested program libraries (see Figure 2), using the floor case throughout. Starting from the base library Σ, we remove `CupodBot` to obtain the seven-program library, or add `CupodTrollBot` and `DupocTrollBot` to obtain the ten-program library. Adding the implication provers `CIMCIC` and `DIMDID` gives the twelve-program library.

In all four libraries, the three equilibrium components yield mutual cooperation with probabilities 1, $11/18$, and 0. The choice of library matters more for equilibrium selection. Every sampled population reached full cooperation in the seven-, eight-, and twelve-program libraries. Adding only the trolls left the support of the fully cooperative component unchanged, yet a small number of populations spent long periods near full defection. `DupocTrollBot` gives `DefectBot` a payoff advantage over `DupocBot`. It cooperates even against the unconditional defection of `DefectBot`, which earns 2 in that encounter, but defects against `DupocBot`, which earns -1. Once `DupocBot` is rare in a population otherwise composed of `DefectBot` and `OBot`, it earns a positive payoff only when it meets another copy of itself. Such encounters are rare, so its share recovers slowly. Even under adversarial pressure from `DupocTrollBot`, the `DupocBot` equilibrium is reached a super-majority of times.

F Experiments and Results on Partial Transparency

Throughout this Appendix we consider the setup given in Section 2.2.1, where a base program `Bot` in $G_{\mathrm{PD}}^{\mathrm{prog}}$ is extended to a family of $\texttt{TauBot}(\alpha)$ in G_{PD}^{τ}, where $\alpha \in [0, 1]$ is the linear threshold parameter and τ is a transparency map, common among the two players.

F.1 Experimental Setup

The setup has two ingredients: a family of transparency maps indexed by a temperature (section F.1.1), and a common scale on which maps from different families can be compared (section F.1.2).

F.1.1 Transparency Maps. We build transparency maps from a notion of distance between programs: the belief a program receives about its opponent concentrates on the programs closest to the one actually submitted.

Fix a finite program library $\Sigma = \{\pi_1, \dots, \pi_N\}$ of admissible programs and assume that $\mathrm{outcome}(\pi_j, \pi_\ell)$ is known for every ordered pair. We consider softmax-based transparency maps induced by a pseudo-distance on Σ.

Definition F.1 (Softmax-based transparency maps). Let $d : \Sigma \times \Sigma \to \mathbb{R}_{\geq 0}$ be a pseudo-distance[25]. For a temperature $T > 0$, the transparency map $\tau_d^T : \Sigma \to \Delta(\Sigma)$ is the softmax of $-d/T$:

$$\tau_d^T(\pi_j)(\pi_\ell) := \frac{\exp\big(-d(\pi_j, \pi_\ell)/T\big)}{\sum_{m=1}^{n} \exp\big(-d(\pi_j, \pi_m)/T\big)}, \qquad \pi_j, \pi_\ell \in \Sigma.$$

As is usual, we extend to $T = \infty$ by $\tau_d^\infty(\pi_j) = \mathrm{unif}_\Sigma$ and to $T = 0$ by $\tau_d^0(\pi_j) = \delta_{\pi_j}$, the full transparency of Section 2.1.

Throughout the experiments, we use two different pseudo-distances that yield the behavioral and syntactic families:

Behavioral Family. For each program $\pi \in \Sigma = \{\pi_1, \dots, \pi_N\}$, define its outcome row as

$$\mathrm{row}(\pi) := \big(\mathrm{outcome}(\pi, \pi_m)_1\big)_{m=1}^{N} \in \{C, D\}^N,$$

so that the mth entry records the action that π takes against π_m. We define the behavioral pseudo-distance as the Hamming distance between two outcome rows, that is, the number of opponents against which the two programs take different actions:

$$d_{\mathrm{bvr}}(\pi_j, \pi_\ell) := \big|\{m \in \{1, \dots, N\} : \mathrm{row}(\pi_j)_m \neq \mathrm{row}(\pi_\ell)_m\}\big| \in \{0, \dots, N\}.$$

Distinct programs can have identical outcome rows and hence have distance zero. For each $T \in [0, +\infty]$, we write τ_{bvr}^T for the transparency map induced by d_{bvr} according to Definition F.1.

Syntactic Family. For each program $\pi \in \Sigma$, let $\mathrm{ast}(\pi)$ denote the abstract syntax tree of its Lean source code, with nodes labeled by the corresponding `Prog` or `Formula` constructors, the action constants C and D, and the names of referenced programs; budget arguments are omitted. For a tree t, let $|t|$ denote its number of nodes. We define the syntactic pseudo-distance as the Zhang-Shasha tree edit distance (K. Zhang and Shasha 1989) between two such trees, normalized by the number of nodes in the larger tree:

$$d_{\mathrm{syn}}(\pi_j, \pi_\ell) := \frac{\mathrm{ted}\big(\mathrm{ast}(\pi_j), \mathrm{ast}(\pi_\ell)\big)}{\max\big\{|\mathrm{ast}(\pi_j)|, |\mathrm{ast}(\pi_\ell)|\big\}},$$

where ted is the minimum number of node insertions, deletions, and relabelings needed to transform one tree into the other, with each operation assigned unit cost. Programs that differ only in their budget arguments have identical trees and hence have distance zero. For each $T \in [0, +\infty]$, we write τ_{syn}^T for the transparency map induced by d_{syn} according to Definition F.1.

As shown in Section 2.2.2, the setup is flexible enough to cover the full spectrum of transparency, from fully transparent OSGT to the base game. In particular, the behavioral family compares the actions that programs take against each opponent in the library, whereas the syntactic family compares the abstract syntax trees of their Lean source code.

[25] A pseudo-distance is a nonnegative, symmetric distance function that is zero from any point to itself and satisfies the triangle inequality.

F.1.2 Transparency Levels. The same temperature T can yield different degrees of transparency in the behavioral and syntactic families because the two pseudo-distances measure different features of a program. To compare the families on a common scale, we measure the fraction of uncertainty about the opponent's identity that a signal drawn from the transparency map removes on average. Formally, let $N = |\Sigma| \geq 2$, draw the opponent's program B uniformly from Σ, and, conditional on $B = \pi$, draw a signal S from $\tau(\pi)$. We define the transparency level of τ as

$$t(\tau) := \frac{I(B;S)}{H(B)} = \frac{H(B) - H(B \mid S)}{H(B)} = 1 - \frac{H(B \mid S)}{\log_2 N} \in [0,1],$$

where $H(B) = \log_2 N$ is the initial uncertainty about the opponent's identity[26], measured in bits, and $H(B \mid S)$ is the uncertainty that remains after observing the signal, averaged over possible signals. Thus, $t(\tau) = 0$ when the signal is independent of the opponent's identity, and $t(\tau) = 1$ when the signal determines that identity. We compare the two families by choosing a temperature for each that yields the same transparency level. At a matched level, the signals remove the same fraction of uncertainty on average, although they may distinguish different pairs of programs.

F.2 Equilibrium under Partial Transparency

Table 11 reports the program-equilibrium components of the eight-program library under behavioral transparency, with hinge cells evaluated on the negative side as in Section 5.1. Every equilibrium in a component yields the same probability of mutual cooperation, $\Pr[(C,C)]$. Reading across a row shows which outcomes become sustainable as transparency increases at fixed caution. Mutual defection remains an equilibrium throughout.

Table 11. Equilibrium outcomes under behavioral transparency. DEF (white) denotes mutual defection alone. BOTH (orange) denotes mutual defection and full cooperation. ALL (yellow) also includes partial cooperation with $\Pr[(C,C)] = 11/18$. The body shows every tenth grid point. The last two columns give the first appearances of full cooperation (t^*) and partial cooperation (t^{**}) on the full grid with spacing 0.01.

	Transparency t											Thresholds	
α	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1	t^*	t^{**}
0.50	DEF	DEF	BOTH	ALL	ALL	ALL	ALL	ALL	ALL	ALL	ALL	0.11	0.28
0.60	DEF	DEF	DEF	BOTH	ALL	ALL	ALL	ALL	ALL	ALL	ALL	0.24	0.39
0.70	DEF	DEF	DEF	DEF	BOTH	ALL	ALL	ALL	ALL	ALL	ALL	0.37	0.50
0.80	DEF	DEF	DEF	DEF	DEF	BOTH	BOTH	ALL	ALL	ALL	ALL	0.50	0.69
0.90	DEF	DEF	DEF	DEF	DEF	DEF	DEF	BOTH	BOTH	ALL	ALL	0.67	0.88
1.00	DEF	DEF	DEF	DEF	DEF	DEF	DEF	DEF	DEF	DEF	ALL	1.00	1.00

At $\alpha = 0.8$, for example, full cooperation first appears at $t = 0.50$. The partially cooperative component appears later, at $t = 0.69$, and coexists with both mutual defection and full cooperation thereafter. At $\alpha = 1$, the two thresholds coincide. Both cooperative outcomes appear only at full transparency.

Whenever full cooperation is an equilibrium outcome, all simulated populations converge under replicator dynamics to a fully cooperative equilibrium. This holds even in the ALL cells, where three equilibrium outcomes coexist. Figure 3 shows that the selected probability of mutual cooperation coincides with the maximum across

[26]The uniform prior reflects a principle of indifference: in the absence of information favoring one program over another, we assign equal probability to every program in Σ. This assumption may be inappropriate when some programs are known to occur more frequently than others. In that case, the same ratio $I(B;S)/H(B)$ can be evaluated under a prior that reflects those frequencies.

program equilibria. Figure 4(a) shows the switch at each caution level. Every run converges to mutual defection below t^* and to full cooperation at and above it.

G Definitions of the Programs

This appendix defines the twelve programs in Table 7, constant programs, simulation-based programs, and proof-based programs, followed by the two trolls and the two implication-based programs. Throughout, π denotes the opponent's program, and $\text{run}(\pi\|\rho)$ denotes the action returned by π when given ρ as its opponent.

For proof-based programs, $k \in \mathbb{N}$ is the proof-length bound, and $\texttt{proof_search}(k, \pi, \varphi)$ succeeds exactly when S has a proof of φ of length at most k. All proof-based programs in a matchup use the same k, as in Table 7.

G.1 Constant Programs

CooperateBot. `CooperateBot` always cooperates, regardless of its opponent's program. It also serves as the unconditional cooperation probe used by the simulation-based programs.

```
def CooperateBot(π):
    return C
```

DefectBot. `DefectBot` always defects, regardless of its opponent's program. It also serves as the unconditional defection probe used by the simulation-based programs.

```
def DefectBot(π):
    return D
```

G.2 Simulation-Based Programs

TitForTatBot. `TitForTatBot` simulates its opponent against `CooperateBot` and returns the same action. Thus it responds to the opponent's treatment of an unconditional cooperator, without requiring a history of repeated play.

```
def TitForTatBot(π):
    if run(π‖CooperateBot) = C:
        return C
    else:
        return D
```

DBot. `DBot` simulates its opponent against `DefectBot` and returns the opposite action. It defects against opponents that cooperate with an unconditional defector, and cooperates with those that defect against it.

```
def DBot(π):
    if run(π‖DefectBot) = C:
        return D
    else:
        return C
```

OBot. `OBot` cooperates only if its opponent cooperates with both `CooperateBot` and `DefectBot`. It tests these responses in that order, defecting as soon as either probe returns D.

```
def OBot(π):
    if run(π‖CooperateBot) = C:
```

```
        if run(π‖DefectBot) = C:
            return C
        else:
            return D
    else:
        return D
```

EBot. `EBot` first defects if its opponent cooperates with `DefectBot`. Otherwise, it cooperates if the opponent cooperates with `CooperateBot` or, failing that, with `MirrorBot`; if neither probe returns C, it defects.

Here `MirrorBot` is an auxiliary probe that simulates its own opponent against `MirrorBot` and returns the resulting action. It is used within `EBot`'s definition and is not an additional member of the twelve-program library.

```
def MirrorBot(ρ):
    return run(ρ‖MirrorBot)

def EBot(π):
    if run(π‖DefectBot) = C:
        return D
    elif run(π‖CooperateBot) = C:
        return C
    elif run(π‖MirrorBot) = C:
        return C
    else:
        return D
```

G.3 Proof-Based Programs

DupocBot. `DupocBo` ("Defect Unless Proof of Cooperation") searches for a proof that its opponent cooperates against it. It cooperates if it finds such a proof within length k, and defects otherwise.

```
def DupocBot(k)(π):
    if proof_search(k, π, run(π‖DupocBot(k)) = C):
        return C
    else:
        return D
```

CupodBot. `CupodBot` ("Cooperate Unless Proof of Defection") searches for a proof that its opponent defects against it. It defects if it finds such a proof within length k, and cooperates otherwise.

```
def CupodBot(k)(π):
    if proof_search(k, π, run(π‖CupodBot(k)) = D):
        return D
    else:
        return C
```

G.4 Troll Programs

The equalities in the following proof searches express syntactic identity of programs, rather than behavioral equivalence.

CupodTrollBot. `CupodTrollBot` searches for a proof that its opponent is exactly `CupodBot`(k). It defects if it finds such a proof within length k, and cooperates otherwise.

```
def CupodTrollBot(k)(π):
    if proof_search(k, π, π = CupodBot(k)):
        return D
    else:
        return C
```

DupocTrollBot. `DupocTrollBot` searches for a proof that its opponent is exactly `DupocBot`(k). It defects if it finds such a proof within length k, and cooperates otherwise.

```
def DupocTrollBot(k)(π):
    if proof_search(k, π, π = DupocBot(k)):
        return D
    else:
        return C
```

G.5 Implication-Based Programs

CIMCIC.. `CIMCIC` ("Cooperate If My Cooperation Implies Cooperation") searches for a proof that its own cooperation implies its opponent's cooperation in the current encounter. It cooperates if it finds such a proof within length k, and defects otherwise.

```
def CIMCIC(k)(π):
    φ = (run(CIMCIC(k)‖π) = C
         ⟹ run(π‖CIMCIC(k)) = C)
    if proof_search(k, π, φ):
        return C
    else:
        return D
```

DIMDID.. `DIMDID` ("Defect If My Defection Implies Defection") searches for a proof that its own defection implies its opponent's defection in the current encounter. It defects if it finds such a proof within length k, and cooperates otherwise.

```
def DIMDID(k)(π):
    φ = (run(DIMDID(k)‖π) = D
         ⟹ run(π‖DIMDID(k)) = D)
    if proof_search(k, π, φ):
        return D
    else:
        return C
```

H Definitions of the Tau Programs

The eight tau programs are obtained by applying a common construction to the definitions in Appendix G. Let `Bot` range over `CooperateBot`, `DefectBot`, `TitForTatBot`, `DBot`, `OBot`, `EBot`, `DupocBot`, and `CupodBot`. We denote its lift by `TauBot`(k, α), where $k \in \mathbb{N}$ is the common proof-length bound and $\alpha \in [0, 1]$ is the caution parameter. The individual names are obtained by prefixing `Tau` to the base name, as in `TauDupocBot`. For uniformity, we

retain k in the notation for every lift, although only the lifts of `DupocBot` and `CupodBot` invoke proof search directly.

The single-hypothesis decision procedure. For a candidate opponent π', let $B^{\delta}_{k,\alpha}$ be the decision procedure obtained from `Bot` by the following substitutions:

(1) The opponent reference denotes π', and every reference to the program itself denotes $\texttt{TauBot}(k, \alpha)$.
(2) Every named probe `Bot'` is replaced by its corresponding lift $\texttt{TauBot'}(k, \alpha)$. Thus a call that runs the opponent against `Bot'` becomes
$$\mathrm{run}\left(\pi' \| \delta_{\texttt{TauBot'}(k,\alpha)}\right).$$
When `Bot'` is a self-reference, its lift is $\texttt{TauBot}(k, \alpha)$.
(3) The same substitutions are made inside formulas passed to `proof_search`. Constants, branch order, and proof-search budgets are preserved: simulations remain simulations, and proof searches remain proof searches.

Here δ_ρ is the point-mass belief assigning probability one to program ρ.

Aggregation. On input belief μ, $\texttt{TauBot}(k, \alpha)$ executes the entire single-hypothesis procedure for each π' in the support of μ. It cooperates exactly when the total probability assigned to hypotheses for which that procedure returns C is at least α.

```
def TauBot(k, α)(μ):
    if α = 0:
        return C
    mass = 0
    for π' in supp(μ):
        if run(B^δ_{k,α} ‖ π') = C:
            mass = mass + μ(π')
    if mass ≥ α:
        return C
    else:
        return D
```